8

Modelagem de classes de projeto

A perfeição (no projeto) é alcançada, não quando não há nada mais para adicionar, mas quando não há nada mais para retirar.

– ERIC RAYMOND, THE CATHEDRAL AND THE BAZAAR

Este capítulo descreve algumas das transformações pelas quais passam as classes e suas propriedades (atributos, operações e associações) com o objetivo de transformar o modelo de classes de análise no modelo de classes de projeto. Neste capítulo, também estudamos outros elementos do diagrama de classes que não são considerados no modelo de análise e que são necessários à construção do modelo de projeto. Esses elementos permitem adicionar mais rigor ao diagrama de classes do sistema. Por exemplo, podemos definir o tipo de cada atributo, a assinatura de cada operação, a navegabilidade de cada associação etc. É também objetivo deste capítulo descrever conceitos que surgem durante a modelagem de classes de projeto, tais como classes abstratas, *interfaces*, polimorfismo e padrões de projeto.

8.1 Transformação de classes de análise em classes de projeto

Esta seção descreve as transformações e os refinamentos que classes de fronteira, de controle e de entidade sofrem na passagem para o modelo de classes de projeto.

8.1.1 Especificação de classes de fronteira

Classes de fronteira devem apenas servir como um ponto de captação de informações a partir do ambiente, ou de apresentação de informações que o sistema (modelo) processou. Portanto, um primeiro ponto importante acerca da especificação de classes de fronteira é que não devemos atribuir a essas classes responsabilidades relativas à lógica do negócio. A única inteligência que essas classes devem ter é a que lhes permite realizar a comunicação com o ambiente do sistema. Há diversas razões para isso. Em primeiro lugar, se o sistema tiver que ser implantado em outro ambiente, as modificações resultantes sobre seu funcionamento propriamente dito (isto é, excluindo as interações com o novo ambiente) seriam mínimas. Além disso, o sistema pode dar suporte a diversas formas de interação com seu ambiente (por exemplo, uma interface gráfica e uma interface de texto). Finalmente, essa separação resulta em uma melhor *coesão* (ver Seção 7.5.2) das classes de fronteira.

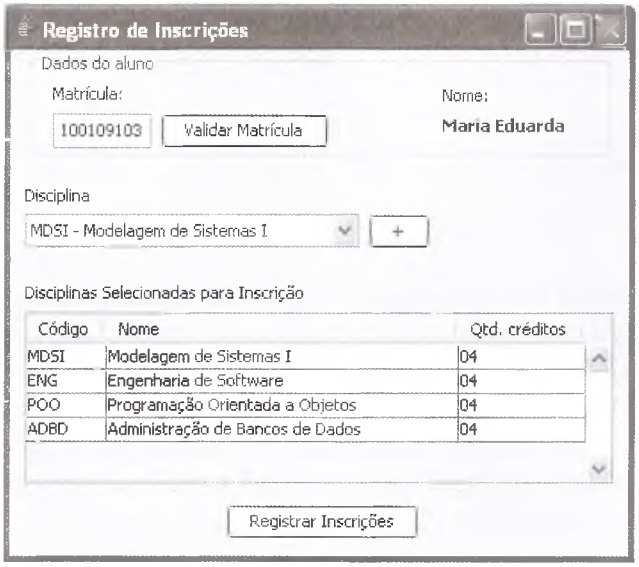


Figura 8-1: Classe de fronteira para interação com o usuário: formulário de inscrição.

Outro aspecto importante é que, durante a análise, considera-se que há uma única classe de fronteira para cada ator que participa de um caso de uso. Na passagem para o modelo de classes de especificação, cada classe de fronteira pode gerar várias outras classes. Nessa transformação, devemos considerar os diferentes tipos de classes de fronteira: classes de interface com o usuário, classes de interface com equipamentos e classes de interface com outros sistemas.

A seguir enumeramos as principais formas de interação com o ambiente que podem existir em um sistema de software. Para todas elas, o projeto de classes

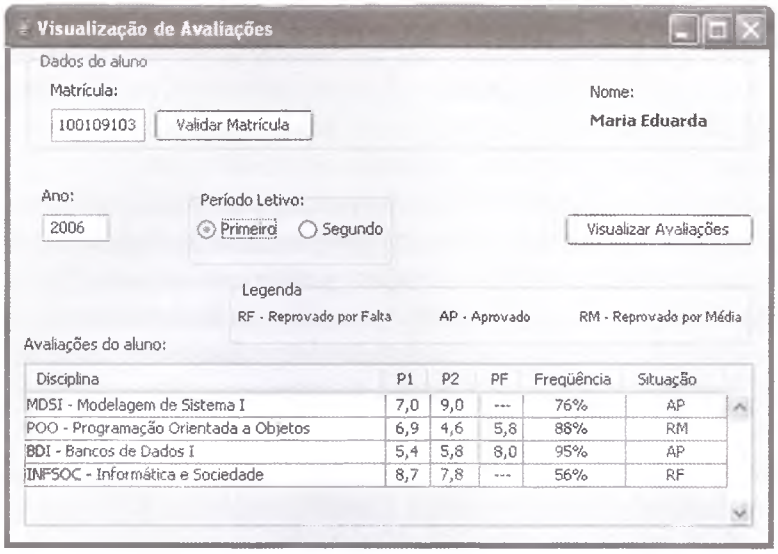


Figura 8-2: Classe de fronteira para interação com o usuário: visualização de avaliação.

de fronteira é necessário. É importante também notar que as formas necessárias de interação do sistema com seu ambiente influenciam no projeto arquitetural desse sistema. Descrevemos o projeto arquitetural no Capítulo 11.

1. **Clientes WEB clássicos.** Essa é a forma mais comum de interação em aplicações WEB. A classe de fronteira é representada por páginas HTML. Muitas vezes, essas páginas são dinâmicas, ou seja, são geradas pelo sistema em função de estímulos que este recebe de seu ambiente. Note que, neste caso, embora as fronteiras do sistema sejam representadas como classes na fase de análise, na fase de projeto essas classes são realizadas como páginas HTML.
2. **Clientes móveis.** É cada vez mais comum o uso da tecnologia móvel em todas as áreas de negócio. Telefones celulares e computadores manuais são cada dia mais freqüentes e interativos. Para esse tipo de interação com o ambiente, normalmente são necessárias classes de fronteira que implementem algum protocolo específico com o ambiente. Um exemplo é a WML (*Wireless Markup Language*).
3. **Clientes stand-alone.** É possível também que a aplicação tenha que fornecer uma interface gráfica por meio de formulários, com botões, menus etc. Nesse caso, é recomendável que os desenvolvedores pesquisem os recursos fornecidos pelo ambiente de programação sendo utilizado. Muitos desses ambientes fornecem componentes para construção de interfaces gráficas. Um exemplo disso é o Swing/JFC da linguagem Java.

4. **Serviços WEB.** Outra tecnologia cujo uso está se tornando bastante comum é a de serviços WEB (*WEB services*). Um serviço WEB é uma forma de permitir que uma aplicação forneça seus serviços (funcionalidades) pela Internet.

Nos casos 1 e 3, as classes de fronteira correspondem à interface como seres humanos. Uma atividade importante de um processo de desenvolvimento que envolve essas classes é o *projeto da interface gráfica com o usuário*. Nessa atividade, são especificados aspectos de aparência e disposição dos componentes gráficos, assim como de navegação pelos diversos formulários da aplicação. Conforme mencionamos na Seção 6.5, o projeto de interface gráfica com o usuário está fora do escopo deste livro.

Nos casos 2 e 4, o sistema deve se comunicar com outro sistema de software ou com um dispositivo (equipamento). Como exemplo, em nosso estudo de caso do Sistema de Controle Acadêmico (ver Seção 4.7), temos um ator denominado Sistema de Faturamento, correspondente a um sistema externo. Nesse estudo de caso, o SCA deve se comunicar com o Sistema de Faturamento, um sistema externo. Nesse caso, podemos criar uma classe denominada SistemaFaturamento para encapsular o comportamento de interação com o sistema externo. Entretanto, é também possível a situação em que a interação com um sistema externo é tão complexa que não seja adequado encapsulá-la em uma única classe. Nesse caso, é necessário substituir a classe de fronteira identificada na fase de análise por um *subsistema*. Este último serve para representar a comunicação com o sistema externo. Além disso, esse subsistema agrupa todas as classes necessárias nessa comunicação e provê uma *interface* (ver Seção 8.5.4) a partir da qual seus serviços podem ser obtidos. (Subsistemas são descritos do ponto de vista arquitetural no Capítulo 11.). A substituição de uma classe de fronteira por um subsistema normalmente é feita com o uso do padrão de projeto *Façade* (ver Seção 8.7.6).

8.1.2 Especificação de classes de entidade

As classes de entidade são aquelas que representam os conceitos do domínio que o sistema deve processar. Essas classes representam as informações e as regras de negócio que direcionam a manipulação dessas informações. Em nosso estudo de caso, exemplos de classes de entidade são `Aluno`, `Professor`, `OfertaDisciplina` etc. Classes de entidade também são chamadas de classes do negócio. A seguir, algumas dicas que devem ser seguidas durante a especificação das responsabilidades de uma classe de entidade.

A maioria das classes de entidade normalmente permanece na passagem da análise ao projeto. Na verdade, classes de entidade são em geral as primeiras classes a serem identificadas, na análise de domínio (ver Seção 2.1.2).

Durante a fase de projeto, outro aspecto importante a considerar sobre classes de entidade é identificar quais delas geram objetos que devem ser *persistentes*. Para essas classes, o seu mapeamento para algum mecanismo de armazenamento persistente deve ser definido. No Capítulo 12, tratamos da questão da persistência de objetos, enfocando o caso no qual o mecanismo de armazenamento é um *sistema de gerenciamento de bancos de dados relacional* (SGBDR).

Um aspecto importante durante o refinamento do modelo de classes de projeto e relativo às classes de entidade é a forma utilizada para representar os relacionamentos (associações, agregações e composições) entre seus objetos. Essa representação é função da *navegabilidade* e da *multiplicidade* definidas para a associação. Descrevemos a especificação de associações na Seção 8.4.

Outro aspecto relevante durante a especificação de uma classe de entidade diz respeito ao modo como podemos identificar cada um de seus objetos *unicamente*. Por exemplo, um objeto da classe Aluno é unicamente identificado pelo valor de sua matrícula (um atributo do domínio). A manipulação dos diversos atributos identificadores possíveis em uma classe pode ser bastante trabalhosa. Para evitar isso, normalmente os projetistas optam por criar um *identificador de implementação*, que não tem correspondente com atributo algum do domínio da aplicação. Uma vantagem dessa abordagem é a possibilidade de manipular identificadores de maneira uniforme e eficiente. Identificadores de implementação são também úteis quando esses objetos de entidade devem ser mapeados para um mecanismo de armazenamento persistente, particularmente para um SGBDR (sobre esse aspecto, ver Seção 12.1.2). Um tipo de dado normalmente utilizado para representar identificadores de implementação é o inteiro. Em *Java, por exemplo, temos os tipos de dados short, int e long, todos possíveis de* serem utilizados para representar identificadores.

8.1.3 Especificação de classes de controle

Assim como as classes de fronteira, classes de controle são normalmente identificadas na modelagem de classes da aplicação. Portanto, controladores normalmente não são objetos do domínio, mas, sim, da aplicação, cuja responsabilidade é coordenar a *interação* entre outros objetos. Objetos controladores são, segundo Craig Larman, *fabricações puras* (ver Seção 7.5.3).

Na atividade de projeto do modelo de classes, devemos refinar cada classe de controle identificada durante a análise. Normalmente, essa classe deve ser particionada em duas ou mais outras classes para controlar diversos aspectos da solução. Isso é feito com o objetivo de evitar a criação de uma única classe com baixa *coesão* e alto *acoplamento* (ver Seção 7.5.2). Alguns exemplos dos aspectos de uma aplicação cuja *coordenação* (e não a *realização* propriamente dita) é de responsabilidade das classes de controle são preenchimento de controles da in-

terface gráfica, autenticação de usuários, controle de acesso a funcionalidades do sistema.

Um tipo comum de objeto controlador é o *controlador de caso de uso*, ao qual já fizemos menção na Seção 5.4.2.3. Esse objeto é responsável pela coordenação da realização de um caso de uso em particular. Mais especificamente, as seguintes responsabilidades são esperadas de um típico controlador de caso de uso:

- Deve coordenar a realização de um caso de uso do sistema.
- Deve servir como canal de comunicação entre objetos de fronteira e objetos de entidade.
- Deve comunicar-se com outros controladores, quando necessário.
- Deve mapear ações do usuário (ou atores de uma forma geral) para atualizações ou mensagens a serem enviadas a objetos de entidade.
- Deve estar apto a manipular exceções provenientes das classes de entidades.

Em aplicações WEB, é comum a prática de utilizar outro tipo de objeto controlador chamado *front controller* (FC). Um FC é um controlador responsável por receber todas as requisições de um cliente (por exemplo, de um navegador WEB ou de um aplicativo móvel). Esse objeto, então, identifica qual o controlador adequado para processar a requisição, possivelmente mediante consulta a algum serviço de mapeamento de URLs para controladores específicos (ou seja, controladores de caso de uso). Uma vez identificado o controlador de caso de uso adequado para processar a requisição, o FC a despacha para ele. O controlador de caso de uso, por sua vez, coordena a execução do caso de uso na produção da resposta esperada. Essa coordenação envolve o envio de mensagens para a camada de objetos de negócio (que podem eles próprios enviar mensagens para outros objetos de entidade), para outros objetos controladores existentes no caso, ou para objetos mediadores (ver mais adiante). Sendo assim, um FC é um ponto central de entrada para as funcionalidades do sistema. Essa estratégia torna mais fácil controlar alguns aspectos da aplicação, como, por exemplo, a autenticação dos usuários. Um objeto FC é um caso especial de controlador fachada, pelo fato de que sua tarefa é recepcionar requisições provenientes dos objetos de fronteira do sistema.

Embora a definição de um controlador para a aplicação como um todo seja típica de aplicações WEB, nada impede que o mesmo esquema de colaboração entre objetos seja utilizado em aplicações convencionais (ou seja, para o *desktop*). Por exemplo, em aplicações do domínio de jogos, é comum a criação de um objeto que representa a aplicação como um todo e que é responsável pela recepção dos eventos do sistema.

Um controlador de caso de uso pode aumentar substancialmente o *acoplamento* (ver Seção 7.5.2) do sistema, visto que ele serve de intermediário entre os

objetos de fronteira e de entidade. Além disso, controladores têm o potencial de levar à duplicação de código. Por exemplo, considere o nosso estudo de caso, o SCA. Nesse sistema, considere que dois casos de uso precisam apresentar ao usuário uma lista de disciplinas. Nessa situação, os controladores desses casos de uso teriam comportamento (código) duplicado para produzir essa lista de disciplinas. É portanto importante que o projetista dispense o tempo que for necessário para evitar o projeto de controladores que apresentem essas características ruins (o acoplamento excessivo e a duplicação de código). Uma alternativa para solucionar esses problemas é a definição de *classes adicionais*, cujo objetivo é diminuir o acoplamento do controlador, uma vez que servem de auxiliares do mesmo durante a execução do caso de uso. Através do uso dessas classes adicionais, o controlador fica livre de realizar certas tarefas por si próprio. Em vez disso, ele delega a realização da tarefa para outro objeto. Essa estratégia também tem a vantagem de permitir que esses objetos auxiliares sejam reutilizados em outros casos de uso do sistema nos quais a mesma tarefa seja necessária novamente. Objetos auxiliares normalmente não têm correspondente no domínio do negócio (ou seja, são *fabricações puras*). No entanto, são úteis, pois tornam o sistema mais flexível, manutenível e resultam em componentes mais reutilizáveis. Na próxima seção, descrevemos mais detalhadamente diversos aspectos do desenvolvimento que demandam a criação (ou a reutilização) de classes adicionais.

8.1.4 Especificação de outras classes

Durante a fase de análise, faz sentido pensarmos somente em objetos pertencentes à categorização BCE (ver Seção 5.4.2.1). Entretanto, um aspecto importante acerca da fase de projeto é que devemos definir todas as características do sistema que sejam necessárias à implementação. Quando chegamos a essa fase, fica claro que só objetos dessas classes não são suficientes para prover as funcionalidades que se espera de um sistema de software. Portanto, a identicação de classes não é uma tarefa exclusiva da fase de análise. Na fase de projeto também devemos nos preocupar em identificar classes para completar as funcionalidades já fornecidas pelas classes provenientes da análise. Nesse contexto, diversos aspectos precisam ser levados em consideração. Alguns exemplos desses aspectos para os quais precisamos definir classes são: distribuição do sistema por diversos nós de processamento e conseqüente necessidade de comunicação entre suas partes, segurança, autenticação, autorização, controle de transações, registro de operações realizadas, armazenamento persistente de informações, leitura de parâmetros de configuração a partir de arquivos no sistema operacional etc. Cada um desses aspectos implica responsabilidades do sistema, para as quais o projetista pode definir suas próprias classes, ou pode utilizar classes preexistentes definidas em *bibliotecas de classes*, *padrões de projeto* ou *frameworks*. No restante desta seção, descrevemos alguns desses aspectos.

Quando falamos em uma *aplicação não-distribuída*, uma requisição de um objeto a outro não ultrapassa os limites do nó de processamento no qual o sistema está hospedado. Entretanto, quando falamos em uma *aplicação distribuída* de uma, queremos dizer que as partes (subsistemas) componentes desta estão fisicamente separadas em diferentes nós de processamento. No caso de haver distribuição, surge o seguinte complicador: como dois objetos residentes em partes fisicamente separadas da aplicação podem trocar mensagens? Em particular, é uma situação comum os objetos de fronteira correspondentes à interface com usuários, e objetos de controle serem alocados em nós de processamento diferentes. Uma alternativa para resolver esse problema é com o uso de *classes procuradoras* (tradução para *proxy classes*). Classes procuradoras são aquelas cuja responsabilidade é fazer com que mensagens entre objetos cruzem as fronteiras entre os nós de processamento através de uma rede de comunicação. O que acontece é que um objeto residente em um nó de processamento pode enviar uma mensagem para um procurador. Este, por sua vez, cuida de repassar essa mensagem para um objeto residente em outro nó de processamento, *de forma transparente para o remetente*. Ou seja, para o objeto remetente, tudo se passa como se o receptor estivesse residente no nó de processamento local. Dois exemplos de tecnologias que podem ser utilizadas para implementar classes procuradoras são RMI (sigla para *Remote Method Invocation*) e CORBA (sigla para *Common Object Request Broker Architecture*). Uma estratégia de distribuição alternativa para o uso de classes procuradoras é copiar objetos de um nó de processamento para outro. Dessa forma, cópias do mesmo objeto são distribuídas e, assim, cada cópia pode receber mensagens sem que haja necessidade de transporte da requisição pela rede de comunicação (como acontece com o uso de objetos procuradores). Entretanto, a cópia de objetos tem a desvantagem de possibilitar a inconsistência entre as cópias criadas. Descrevemos mais detalhes sobre a distribuição de um sistema na Seção 11.2, quando apresentamos a definição da arquitetura física de um sistema.

É comum, em aplicações que envolvem interface gráfica, a situação em que determinado formulário (objeto de fronteira) é apresentado ao usuário com alguns de seus controles (por exemplo, listas, menus suspensos) já preenchidos com informações provenientes dos objetos de negócio. Para exemplificar esse aspecto, considere a seguinte tarefa comum em uma aplicação: *preenchimento de controles da interface gráfica*. Considere novamente a Figura 8-1, na qual apresentamos o protótipo do formulário para inscrição do SCA. Note que, quando este formulário for apresentado ao usuário, o menu suspenso que apresenta disciplinas disponíveis para inscrição já deve estar preenchido. Nesse caso, o objeto de controle pode delegar a outro objeto a obtenção da lista de disciplinas a ser apresentada no objeto de fronteira. Objetos cuja responsabilidade é obter informações e agregá-las em um formato específico são chamados *View Helpers* (*VH*). Uma classe VH é, portanto, qualquer classe que au-

xilie na apresentação de informações presentes nas classes de negócio de uma maneira reutilizável.

Outro aspecto importante durante o projeto diz respeito à definição de classes para realizarem o acesso a um mecanismo de armazenamento persistente (por exemplo, um SGBD). Essas classes são necessárias para a obtenção e consolidação de informações provenientes dos objetos de negócio (entidades). Uma alternativa para esse problema de projeto é o uso de *classes DAO* (sigla para *Data Access Object*). Uma classe *DAO* encapsula e implementa operações CRUD (tradução para *create, read, update, delete*) para manipulação de objetos do domínio. Objetos DAO normalmente são criados (instanciados) através de *objetos fábrica* (ver Seção 8.6.4). Entretanto, classes DAO são uma das possíveis alternativas para acesso a um mecanismo de armazenamento persistente. Outras alternativas técnicas são o uso de um SGBD orientado a objetos ou de um framework de persistência. Descrevemos essas técnicas em mais detalhes na Seção 12-2.

Durante o desenvolvimento de um sistema, os projetistas devem usar o máximo de sua experiência e criatividade para definir uma solução de classes a mais adequada possível para o problema em questão. Entretanto, é também comum a reutilização de classes e de soluções preexistentes durante o desenvolvimento. Essas fontes de reutilização devem ser pesquisadas pelos projetistas para evitar o desenvolvimento (codificação) desnecessário. Três fontes de reutilização são *padrões de projeto, bibliotecas de classes* e *frameworks*. Descrevemos essas fontes no que segue.

Um *padrão de projeto* é uma descrição da essência da solução para um problema que ocorre de forma recorrente no desenvolvimento de software. Um padrão de software descreve um punhado de objetos em colaboração para solucionar o problema recorrente. Padrões de software são normalmente identificados e catalogados por desenvolvedores de software experientes. O uso de padrões de software permite que desenvolvedores sejam mais produtivos, pois evita que estes desenvolvedores "reinventem a roda" durante o desenvolvimento: se existe algum padrão de software para um problema com o qual o desenvolvedor se depara, a solução apontada por esse padrão serve como ponto de partida. Descrevemos alguns exemplos de padrões de software na Seção 5.4.4 e na Seção 8.6. Além disso, as classes VH, DAO e Front Controller descritas nesta seção fazem parte dos padrões descritos no catálogo *J2EE* (Alur *et al.*, 2003).

Um *framework* é uma coleção de classes em colaboração, assim como um padrão de projeto. Entretanto, há duas diferenças marcantes entre um framework e um padrão de projeto. Em primeiro lugar, um framework é muito maior (possui uma quantidade maior de classes) que o definido em um padrão. Além disso, um framework apresenta implementação; em um padrão, a solução apontada pelo mesmo deve ser implementada pelo desenvolvedor. Dois exemplos de frameworks de código aberto são o *Hibernate* (*http://www.hibernate.org*)

e o *JUnit* (*http://www.junit.org*). O primeiro é um framework para persistência em um SGBD relacional para Java e .NET. O segundo é um framework para auxiliar o desenvolvedor a definir testes para sua aplicação.

Finalmente, uma *biblioteca de classes* corresponde a uma coleção de classes que possuem algum objetivo específico. Exemplos de bibliotecas de classe são a JDBC e a Swing, existentes na linguagem Java, a primeira para acesso a um SGBD relacional, e a segunda para a construção de interfaces gráficas com o usuário. Outro exemplo de biblioteca de classes bastante popular, dessa vez para realizar o registro de operações da aplicação, é a Log4J (*http://logging.apache.org/log4j/docs/*). As próprias plataformas de desenvolvimento como Java e .NET já fornecem diversas bibliotecas de classes reutilizáveis; a tecnologia RMI que mencionamos anteriormente nesta seção é um exemplo de biblioteca de classes existente em Java.

8.2 Especificação de atributos

8.2.1 Notação da UML para atributos

No modelo de classes de análise, os atributos, quando representados, o são somente pelo nome. No entanto, a sintaxe completa da UML para definição de um atributo é bem mais detalhada. Essa sintaxe, cujo uso é apropriado para a fase de especificação do modelo de classes, é a seguinte:

```
[/]visibilidade nome: tipo = valor_inicial
```

Vamos descrever cada um dos elementos dessa sintaxe. (A Figura 8-3 ilustra a utilização desses elementos para atributos da classe `Cliente`.)

A *visibilidade* de um atributo diz respeito ao seu nível de acesso. Ou seja, a visibilidade permite definir quais atributos de um objeto são acessíveis por outros objetos. A visibilidade de um atributo pode ser *pública*, *protegida*, *privativa* ou *de pacote*. A primeira é a visibilidade por omissão (se não for especificada visibilidade alguma, essa é assumida). O símbolo e o significado de cada visibilidade são apresentados na Tabela 8-1.

Cliente
#nome : String -dataNascimento : Data -telefone : String #/idade : int #limiteCrédito : Moeda = 500.0 -quantidadeClientes : int -idadeMédia : float

Figura 8-3: Especificação dos atributos da classe Cliente.

A propriedade de visibilidade serve para implementar o *encapsulamento* da estrutura interna da classe. Somente as propriedades que são realmente necessárias ao exterior da classe devem ser definidas com visibilidade pública. Todo o resto é "escondido" dentro da classe através das visibilidades protegida e privativa. Um atributo definido com a visibilidade protegida diz respeito ao conceito de herança entre classes. Esse conceito é descrito na seção 8.5.

Tabela 8-1: Possíveis visibilidades de um elemento (atributo ou operação)

Visibilidade	Símbolo	Significado
Pública	+	Qualquer objeto externo pode obter acesso ao elemento, desde que tenha uma referência para a classe em que o atributo está definido.
Protegida	#	O elemento protegido é visível para subclasses da classe em que este foi definido.
Privativa	_	O elemento privativo é invisível externamente para a classe em que este está definido.
Pacote	~	O elemento é visível a qualquer classe que pertence ao mesmo pacote no qual está definida a classe.

O elemento *nome* da sintaxe do atributo corresponde ao nome do atributo. Na verdade, somente esse elemento é obrigatório na sintaxe de declaração de um atributo.

O elemento *tipo* especifica o tipo do atributo. Esse elemento é dependente da linguagem de programação na qual a classe deve ser implementada. O tipo definido para um atributo pode ser definido pela utilização de um *tipo primitivo* da linguagem de programação a ser utilizada na implementação. Por exemplo, se estamos utilizando a linguagem Java, temos, dentre outros, os seguintes tipos primitivos: int, float e boolean. É também comum a utilização de *tipos abstratos de dados* (tradução para *abstract data type*, TAD) para declarar atributos. Um TAD é um tipo definido pelo usuário ou um tipo disponível na linguagem de programação que contém uma estrutura interna. Por exemplo, na Figura 8-3, os atributos `Nome` e `telefone` são definidos como do tipo String, que é um tipo abstrato de dados. Além de String, temos (em Java): List, Set e array. Embora freqüentemente corresponda a uma classe, um TAD é representado no compartimento de atributos em vez de ser representado como uma classe separada no diagrama de classes. Isso porque o diagrama ficaria muito carregado visualmente se essas classes TAD fossem representadas por retângulos separados.

A maioria das linguagens de programação já fornece tipos abstratos de dados. No entanto, os desenvolvedores podem definir seus próprios tipos. Por exemplo, a Figura 8-3 apresenta o atributo `limiteCrédito` como do tipo Moeda,

um TAD para manipular valores monetários. Outros exemplos de TAD são listados a seguir.

- Data (para processamento de datas em geral).
- Horário (para processamento de valores de tempo).
- Endereço (para armazenamento de endereços em geral).
- CódigoPostal, CPF e CNPJ (para armazenamento e validação desses valores).

Um *valor inicial* também pode ser declarado para o atributo. Dessa forma, sempre que um objeto de uma classe é instanciado, o valor inicial declarado é automaticamente definido para o atributo. Assim como o tipo, o valor inicial de um atributo também é dependente da linguagem de programação.

Pode-se definir um *atributo derivado* em uma classe. Um atributo é derivado quando o seu valor pode ser obtido a partir do valor de outro(s) atributo(s). Por exemplo, o valor da idade de uma pessoa pode ser obtido a partir de sua data de nascimento. Um atributo derivado é representado com uma barra inclinada à esquerda. Atributos derivados normalmente são definidos por questões de desempenho. Por exemplo, considere a classe Cliente da Figura 8-3. Em vez de calcular a idade média de toda a coleção de clientes a cada vez que este valor é necessário, definimos o atributo derivado idadeMédia e um método seletor correspondente.

Outra característica de um atributo é o seu *escopo*. Cada objeto tem um valor para cada atributo definido em sua classe. Esse é o caso mais comum, e diz-se que o atributo tem *escopo de objeto*. No entanto, pode haver atributos que tenham *escopo de classe*, ou seja, que armazenam valor comum a todos os objetos da classe. Esse tipo de atributo é definido por um sublinhado na sintaxe da UML. Um atributo que tenha escopo de classe é também denominado ***atributo estático***. Por exemplo, a classe `Cliente` da Figura 8-3 contém o atributo `quantidadeClientes` para armazenar a quantidade de instâncias criadas a partir dessa classe. Há apenas um valor desse atributo para todos os objetos da classe Cliente. O atributo `idadeMédia` também é estático.

Uma possível aplicação dos atributos estáticos é na implementação de regras de negócio. Por exemplo, considere a Figura 8-4, em que a classe `Curso` possui um atributo estático `quantidadeMáximaAlunos`. Este atributo pode ser utilizado para implementação da seguinte regra de negócio: *Em um curso podem estar matriculados até 30 alunos.*

Figura 8-4: Atributos estáticos podem ser utilizados para implementar regras de negócio.

8.3 Especificação de operações

Na descrição do modelo de interações, declaramos que as operações de uma classe são identificadas em conseqüência da identificação de mensagens enviadas de um objeto a outro. No detalhamento dessas operações, devemos considerar diversos aspectos: seu nome, lista de parâmetros, tipo de cada parâmetro, tipo de retorno. Nesta seção, descrevemos a notação fornecida pela UML para realizar esse refinamento de operações. Descrevemos também alguns princípios e dicas recomendados durante essa atividade.

8.3.1 Notação da UML para operações

As operações de uma classe correspondem a algum processamento realizado por essa classe. Em termos de implementação, uma operação é uma rotina associada a uma classe. Conforme visto no Capítulo 5, a construção do modelo de classes de análise permite a identificação de algumas operações. Durante a construção do modelo de interações, essas operações são validadas e várias outras operações são identificadas. Todas essas operações devem ser adicionadas ao diagrama de classes e devem ser documentadas com a definição de sua *assinatura*. A sintaxe definida na UML para a assinatura de uma operação é a seguinte:

```
visibilidade nome(parâmetros): tipo-retorno {propriedades}
```

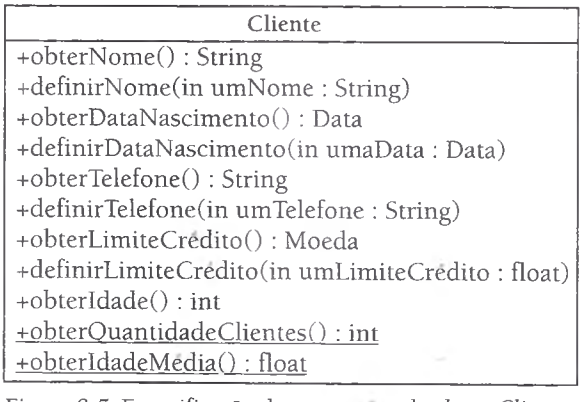


Figura 8-5: Especificação das operações da classe Cliente.

O elemento *nome* corresponde ao nome dado à operação. A *visibilidade* usa a mesma simbologia das visibilidades para atributos. Se uma operação é pública, ela pode ser ativada com a passagem de uma mensagem para o objeto. Se a operação é protegida, ela só é visível para a classe e para seus descendentes (o conceito de *descendente* de uma classe é descrito no Capítulo 9). Finalmente, se a

operação é privativa, somente objetos da própria classe podem executá-la. Os *parâmetros* de uma operação correspondem a informações que esta recebe quando é executada. Normalmente, essas informações são fornecidas pelo objeto remetente da mensagem que requisita a execução da operação no objeto receptor. Uma operação pode ter zero ou mais *parâmetros*. Estes são separados por vírgulas. Cada parâmetro da lista tem a seguinte sintaxe:

```
direção nome-parâmetro: tipo-parâmetro
```

O elemento *direção* serve para definir se o parâmetro pode ou não ser modificado pela operação. Através desse elemento, o modelador pode definir se o parâmetro é de entrada, saída ou ambos. Se a direção for omitida, o valor assumido pelo parâmetro é *in*. Os significados das direções são fornecidos na Tabela 8-2. Note que a direção *in* corresponde à passagem de parâmetros por valor, enquanto as direções *out* e *inout* correspondem à passagem de parâmetros por referência. Essas formas de passagem de parâmetros são comuns à maioria das linguagens de programação, resguardadas as diferenças sintáticas de cada uma.

Tabela 8-2: Possíveis direções na definição de um parâmetro

Direção	Significado definido pela UML
in	Parâmetro de entrada: não pode ser modificado pela operação. Serve somente como informação para o objeto receptor.
out	Parâmetro de saída: pode ser modificado pela operação para fornecer alguma informação ao objeto remetente.
inout	Parâmetro de entrada que pode ser modificado.

O elemento *nome-parâmetro* corresponde ao nome do parâmetro. Cada parâmetro deve ter um nome único dentro da assinatura da operação. O elemento *tipo-parâmetro* corresponde ao tipo do parâmetro e é dependente da linguagem de programação. O elemento *tipo-retorno* representa o tipo de dados do valor retornado por uma operação. Esse tipo depende da linguagem de programação. Assim como atributos, as operações também possuem um *escopo*. Uma operação pode ter *escopo de classe* ou *escopo de instância*. Uma operação que tem escopo de classe processa atributos estáticos. Se essa operação tem escopo de classe, ela é também chamada de *operação estática*. Uma operação que tenha escopo de instância indica que ela processa atributos que têm escopo de instância.

8.3.2 Dicas práticas

Recomenda-se atribuir um nome a uma operação de forma que este lembre o resultado da operação e tenha a forma geral <verbo> + <complemento>. Além dis-

so, o desenvolvedor deve se lembrar de que uma operação é um serviço que a classe presta. Por conseguinte, esse nome deve ser definido a partir da perspectiva externa, e não interna. Ele deve também deixar claro o que a operação produz como resultado, e não como ela faz isso. Por exemplo, considere a Figura 8-5. Essa figura apresenta uma operação de nome `obterIdadeMédia( )`. Esse nome é mais apropriado do que o nome `calcularIdadeMédia( )`, por exemplo.

Note que todas as operações que correspondem a mensagens de um objeto a outro em um diagrama de interação (ver Seção 7.5.1) devem ter visibilidade pública. Isso porque, para um objeto enviar uma mensagem a outro, a operação no objeto receptor deve ser conhecida pelo objeto remetente da mensagem.

O desenvolvedor deve definir operações com o mínimo de parâmetros possível. Uma razão para isso é que quanto menos parâmetros uma operação tiver, mais fácil é a utilização da mesma. Além disso, quanto menor a quantidade de parâmetros de uma operação, menor é a dependência entre as classes envolvidas. É também adequado que o modelador defina, para cada parâmetro, seu valor assumido (se existir).

Da mesma forma que a lista de parâmetros de uma operação deve ser a mínima possível, assim também deve ser a lista de operações que podem modificar o estado de um objeto de certa classe. Dessa forma, o projetista deve avaliar cuidadosamente quais atributos de certa classe podem ser alterados e quais operações são realmente necessárias para realizar a alteração.

Assim como para tipos de atributos e de parâmetros, o tipo de retorno também pode ser um TAD. Nem todas as operações retornam um valor, portanto, esse elemento nem sempre é especificado.

8.3.3 Projeto por contrato

Além de todas as informações descritas no diagrama de classes, o modelador deve também especificar o *objetivo* e o *funcionamento* de cada operação. A descrição do funcionamento, uma descrição da lógica da operação, é útil quando o algoritmo da operação for complexo. O diagrama de atividades da UML, descrito no Capítulo 10, é uma boa ferramenta para realizar essa descrição do funcionamento.

Outra forma de documentar uma operação é com a técnica de *projeto por contrato* (tradução para *Design by Contract, DbC*), proposta por Bertrand Meyer, o criador da linguagem de programação orientada a objetos *Eiffel* (Meyer, 1997). O objetivo da aplicação dessa técnica é obter uma especificação mais rigorosa das operações de uma classe (ou interface) que seja útil para a implementação, o teste e o uso das mesmas. Meyer utiliza o termo *fornecedor* para denotar o desenvolvedor, e o termo *consumidor* para denotar o usuário. Se uma classe A utiliza os serviços de uma classe B, então B é fornecedor e A é o consu-

midor. A idéia básica do projeto por contrato é criar um contrato entre o *desenvolvedor* e os consumidores de uma operação. Para cada parte envolvida no contrato, há obrigações e benefícios. Para um resumo, veja a Tabela 8-3.

No projeto por contrato, há dois elementos, as *invariantes* e as *asserções*. Uma asserção, por sua vez, pode ser de dois tipos: *precondição* ou *pós-condição*. Vamos descrever dois elementos componentes da técnica DbC nos próximos parágrafos.

Uma *precondição* é uma condição atrelada a uma operação e que deve ser satisfeita pelo consumidor quando este requisitar a execução da operação ao fornecedor. A cada operação, é adicionado um conjunto de precondições. Como exemplo de precondição, podemos citar a faixa de valores possíveis para certo parâmetro de uma operação. Além das precondições, para cada operação, também são definidas *pós-condições*. Considere que $C_{pré}$ e $C_{pós}$ são dois conjuntos de precondições e de pós-condições, respectivamente, associados a certa operação. O fornecedor deve garantir que cada pós-condição pertencente a $C_{pós}$ é satisfeita após a execução da operação, contanto que as precondições pertencentes a $C_{pré}$ tenham sido satisfeitas pelo consumidor quando da chamada da operação. Se pelo menos uma das precondições em $C_{pré}$ não forem garantidas pelo consumidor, a execução correta da operação não é garantida pelo fornecedor.

Outro elemento da técnica de DbC é a *invariante*. A cada classe pode ser associado um conjunto de invariantes (note a diferença em relação às asserções, que são associadas a cada operação). Uma invariante de uma classe C é uma condição que deve ser obrigatoriamente satisfeita após quaisquer mudanças realizadas no estado de algum objeto de C. O objetivo de uma invariante é garantir que o objeto permanece em um *estado válido* após qualquer modificação aplicada sobre ele.

Tabela 8-3: Obrigações de benefícios associados a consumidores e a fornecedores no DbC

	Obrigações	Benefícios
Consumidor	Satisfazer precondições	Resultado satisfaz pós-condições e invariantes
Fornecedor	Satisfazer precondições e invariantes	Implementação pode confiar na validade das precondições

A técnica DbC pode ser aplicada formalmente com o uso da Linguagem de Restrição de Objetos (ver Seção 3.6), para definir as asserções de cada operação e invariantes de cada classe. Dois outros exemplos de linguagens de especificação formais são a VDM (*Vienna Development Method*) e a Z.

A definição de asserções e invariantes influencia diretamente na implementação da classe em questão. Mas especificamente, essa influência diz respeito à forma de validar os valores dos parâmetros de suas operações e de seus atribu-

tos. Com relação a isso, um recurso bastante comum em linguagens de programação OO, e que pode ser utilizado para implementar asserções e invariantes, é a *exceção*. Uma exceção normalmente corresponde a uma classe predefinida na linguagem, que pode ser utilizada para notificar alguma situação de erro durante o processamento de uma operação. Quando alguma situação de exceção ocorre em uma operação, um objeto de alguma classe de exceção correspondente ao erro acontecido pode ser criado e retorna a região de código que fez a chamada à operação. Particularmente no caso dos parâmetros de operações e dos atributos de uma classe, um exemplo de situação de exceção é o fato de ser atribuído a algum desses elementos um valor inválido. O Quadro 8-1 apresenta, novamente com a sintaxe da linguagem Java, um exemplo de validação de invariantes da classe `Aluno` com o uso de exceções. Note que essa operação (na verdade, um dos *construtores* da classe) utiliza a exceção chamada IllegalArgumentException para notificar o chamador de que houve algum erro na passagem dos argumentos para a operação. De qualquer forma, é importante que o projetista defina (1) a faixa de valores válidos para cada parâmetro de operação e para o atributo em uma classe e (2) que exceções devem ser disparadas quando a verificação desses valores identificar algum valor inadequado.

Quadro 8-1: Exemplo de validação de invariantes com o uso de exceções

```
// Invariante: sexo in {"M", "F"}
// Invariante: nome é uma cadeia de caracteres não vazia
// Invariante: matrícula é uma cadeia de caracteres não vazia
public class Aluno {
 private String nome;
 private String matrícula;
 private Char sexo;
 ...
public Aluno( final String nome, final String matrícula, final Char sexo) {
  if ( nome == null || nome.equals("") ) {
   throw new IllegalArgumentException("Valor inválido para nome.");
  }
  if ( matrícula == null || matrícula.equals("")) {
   throw new IllegalArgumentException("Valor inválido para matrícula.");
  }
  if ( sexo == null || (!sexo.equals("M") && !sexo.equals("F"))) {
   throw new IllegalArgumentException("Valor inválido para sexo.");
  }
  this.nome = nome;
  this.matricula = matrícula;
  this.sexo = sexo;
 }
 ...
}
```

8.3.4 Operações de criação e destruição de objetos

Duas operações utilizadas freqüentemente nas interações entre objetos são a *criação* (instanciação) e a *destruição* de objetos. Essas operações correspondem às mensagens de criação e destruição de objetos, que descrevemos na Seção 7.2.4. Uma operação de criação serve para instanciar um objeto. Operações de destruição normalmente não têm parâmetros e servem para liberar alguma quantidade de memória que tenha sido alocada dinamicamente na instanciação.

Normalmente há diversas operações de criação definidas em uma classe. Os parâmetros de uma operação de criação são utilizados para iniciar um ou mais atributos do objeto. Para definir os parâmetros de uma operação de criação, considere o seguinte: se o valor do atributo é essencial para o significado do objeto, é mais adequado iniciar o seu valor logo na criação do objeto. Se esse for o caso, uma das operações de criação deve ter um parâmetro que sirva de valor inicial para o atributo essencial. Diz-se também que operações de criação são utilizadas para definir o *estado inicial* de um objeto. O *estado* de um objeto é definido como o conjunto de valores de seus atributos em um dado momento. A modelagem de estados de um objeto é mais bem descrita no Capítulo 9.

A forma de utilização de métodos de criação e destruição varia de uma linguagem de programação para outra. Por exemplo, na linguagem *Delphi* os métodos de criação (*Create*) e de destruição (*Destroy*) são invocados explicitamente. A linguagem *C++* fornece operadores para a criação e destruição de objetos (*new* e *delete*). Já as linguagens *C#* e *Java*, possibilitam a criação de objeto através de um operador (*new*) e possuem um mecanismo automático de remoção (destruição) de objetos, quando esses não são mais necessários à aplicação.

8.3.5 Seletores e modificadores

Pelo princípio do *encapsulamento* (ver Seção 1.2.3.1), qualquer acesso a informações internas a uma classe deve ser realizado através de sua *interface*. A interface de uma classe é definida como o conjunto de suas operações de visibilidade pública.

Para assegurar o encapsulamento da classe, o projetista deve definir todos os seus atributos com visibilidade privativa ou protegida. A seguir, ele deve definir operações para obter acesso e para modificar o valor de cada atributo. Essas operações são denominadas *seletores* e *modificadores*. (Também recebem o nome comum de *métodos de acesso*.) Um *seletor* retorna o valor de um atributo, enquanto um *modificador* altera o valor de um atributo.

Como convenção, o nome de um seletor é formado com o prefixo "obter" seguido do nome do atributo. Analogamente, o nome de um modificador é construído com o prefixo "definir" seguido do nome do atributo. Outra conven-

ção comum é utilizar o nome do atributo para nomear tanto o seu seletor quanto o seu modificador. Como exemplo, considere novamente a Figura 8-5, que apresenta seletores e modificadores para os atributos da classe `Cliente`. Repare que cada seletor possui um tipo de retorno. Já os modificadores não possuem tipo de retorno, mas cada um possui um parâmetro de mesmo tipo do atributo correspondente.

Nem sempre faz sentido ter um modificador para um atributo. Isso acontece quando o valor desse atributo somente deve ser consultado, mas não alterado. Voltando ao exemplo da Figura 8-5, os atributos estáticos só possuem seletores, pois seus valores dependem do número de objetos da classe.

8.3.6 Outras operações típicas

Durante o projeto de uma classe, diversos métodos devem ser definidos. Um exemplo disso são os *métodos de ligação*. Como o próprio nome deixa transparecer, esse tipo de método serve para definir ligações (conexões) entre objetos a tempo de execução do SSOO. Considere o Quadro 8-2, que apresenta uma classe em linguagem Java para representar turmas. Note que essa classe possui um atributo que corresponde a uma coleção de objetos da classe `OfertaDisciplina`. Além disso, note que os métodos de ligação `adicionarOfertaDisciplina` e `removerOfertaDisciplina` estão definidos nessa classe. Esses métodos serverm para criar e destruir conexões entre os objetos das classes `Turma` e `OfertaDisciplina` quando da execução do sistema.

Quadro 8-2: Operações para manutenção de associações

```
public class Turma {
  private Set<OfertaDisciplina> ofertasDisciplina = new HashSet();

  public Turma() {
  }

  public void adicionarOfertaDisciplina(OfertaDisciplina oferta) {
      this.ofertasDisciplina.add(oferta);
  }

  public boolean removerOfertaDisciplina(OfertaDisciplina oferta) {
      return this.ofertasDisciplina.remove(oferta);
  }

  public Set getOfertasDisciplina() {
      return Collections.unmodifiableSet(this.ofertasDisciplina);
  }
}
```

Além dos métodos de ligação, outros métodos comumente encontrados são para calcular valores derivados de atributos da classe, métodos de formatação, métodos de conversão e cópia de objetos. Além disso, alguns métodos que definimos em uma classe devem ter uma operação inversa óbvia. Alguns exemplos: `habilitar` versus `desabilitar`; `tornarVisível` versus `tornarInvisível`; `adicionar` versus `remover`; `depositar` versus `sacar` etc. De qualquer modo, devemos sempre validar a necessidade de cada método com relação ao modelo de interações (ver o Capítulo 7).

8.4 Especificação de associações

Nesta seção, descrevemos alguns detalhes a serem adicionados para refinar os relacionamentos de associação identificados do modelo de classes de análise. Aqui, definimos o conceito de dependência, sua notação, e de que forma esse conceito pode ser utilizado no refinamento de associações.

8.4.1 O conceito de dependência

No modelo de classes de análise, os relacionamentos entre objetos foram definidos apenas com o uso da associação (ou como um de seus casos especiais, a agregação ou a composição). Entretanto, o fato de existir uma associação entre duas classes implica a existência de uma *dependência* entre as mesmas. Uma dependência entre classes indica que uma classe depende dos serviços fornecidos pela outra.

No modelo de análise, é suficiente para o modelador identificar a existência de associações entre classes, que é uma forma de dependência. Mas, na fase de especificação do modelo de classes, essa dependência precisa ser melhor definida pelo projetista, visto que a mesma tem influência na forma utilizada para implementar as classes envolvidas. Sendo assim, vamos então descrever detalhes acerca das possíveis formas de dependência. Para isso, considere duas classes, A e B, onde A depende de B. Vários tipos de dependência entre essas duas classes podem existir. Esses tipos são descritos a seguir.

1. **Dependência por atributo:** A possui um atributo cujo tipo é B. A dependência por atributo é também chamada de *dependência estrutural*.
2. **Dependência por variável global:** A utiliza uma variável global cujo tipo é B.
3. **Dependência por variável local:** A possui alguma operação cuja implementação utiliza uma variável local de tipo B.
4. **Dependência por parâmetro:** A possui pelo menos uma operação, e esta possui pelo menos um parâmetro, cujo tipo é B.

Se analisarmos os tipos de dependência descritos anteriormente, podemos constatar que uma dependência realmente indica que uma classe depende dos serviços fornecidos por uma outra classe. Por exemplo, em uma dependência por parâmetro, possivelmente a implementação da operação em A invoca uma ou mais operações definidas em B.

A notação da UML para representar uma dependência no diagrama de classes é de uma seta tracejada ligando as classes envolvidas. O sentido da seta é da classe dependente para a classe da qual ela depende. A Figura 8-6 ilustra a notação da UML para dependências.

Figura 8-6: Notação para o relacionamento de dependência.

Ainda com respeito à notação, as dependências podem ser estereotipadas para indicar o tipo de dependência existente entre duas classes. A tabela a seguir exibe os estereótipos predefinidos na UML que podem ser utilizados em dependências.

Tabela 8-4: Exemplo de validação de invariantes com o uso de exceções

Estereótipo	Tipo de dependência
`<<global>>`	`Por variável global`
`<<local>>`	`Por variável local`
`<<parameter>>`	`Por parâmetro`

A Figura 8-7 ilustra exemplos de dependências entre classes. Os parâmetros de `oper1` e `oper2` fazem com que `ClasseA` seja dependente de `ClasseB` e de `ClasseC`.

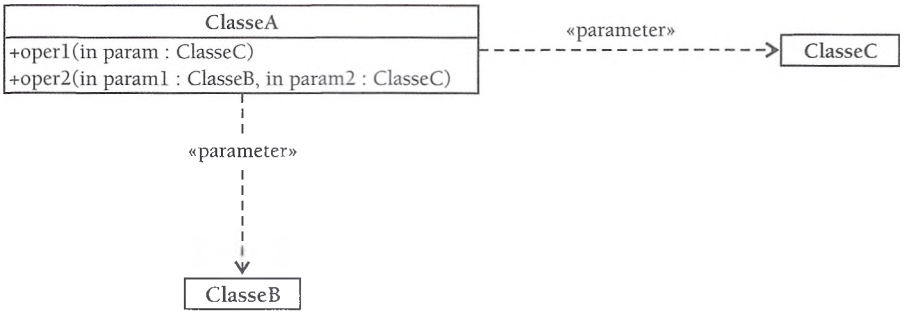


Figura 8-7: Exemplo de uso de dependências em um diagrama de classes.

Agora que definimos o conceito de dependência e vimos sua notação, é importante analisarmos como esse conceito influencia o refinamento de associações provenientes do modelo de classes de análise. Fazemos isso na próxima seção.

8.4.2 Transformação de associações em dependências

No modelo de classes de análise, quando uma nova classe é identificada, o modelador especifica uma ou mais associações entre essa classe e as demais. Na passagem do modelo de classes de análise para o de especificação, o modelador deve estudar cada associação para identificar se ela pode ser transformada em dependências dos tipos 2, 3 ou 4 descritos na Seção 8.4.1. A razão para essa transformação é aumentar o encapsulamento das classes: a dependência por atributo (tipo 1), que corresponde ao modo de implementar associações, torna as classes envolvidas mais dependentes umas das outras. Quanto menos dependências estruturais houver no modelo de classes, maior será o encapsulamento das classes constituintes. Além disso, o acoplamento (ver Seção 7.5.2) do modelo de classes também diminui proporcionalmente à quantidade de dependências estruturais existentes.

Certamente não há um critério predefinido que o modelador pode utilizar para decidir as associações que continuam e as que devem ser transformadas em dependências não-estruturais. Essa decisão depende de vários fatores e varia de caso para caso. No entanto, de um modo geral, as associações que ligam classes de entidade permanecem como associações no modelo de especificação. Isso porque associações entre classes de entidade normalmente devem ser mantidas de forma persistente (ver o Capítulo 12). Além disso, associações entre controladores e classes de entidade são bastante suscetíveis a serem transformadas em dependências não-estruturais (as de tipo 2, 3 ou 4). Entretanto, note que, se um controlador utilizar um objeto de entidade muito freqüentemente, talvez seja mais adequado manter a associação através de uma dependência estrutural, por motivos de desempenho.

Em relação às associações entre classes de fronteira e controladores, o modelador deve considerar o padrão de interação de cada classe de fronteira. Se o caso de uso em questão possuir um ator humano que inicia a sua realização, existirá uma classe de fronteira para esse ator. Essa classe pode manter um relacionamento estrutural com o controlador do caso de uso. Se existirem outras classes de fronteira (para equipamentos ou para outros sistemas, por exemplo), normalmente haverá *interfaces* para essas classes, situação em que é mais apropriado utilizar dependências não-estruturais (o conceito de interface é descrito em detalhes na Seção 8.5.4).

8.4.3 Navegabilidade de associações

As associações (assim como agregações e composições) podem ser classificadas em *bidirecionais* e *unidirecionais*. Uma associação bidirecional indica que há um conhecimento mútuo entre os objetos associados. Ou seja, se um diagrama de classes exibe uma associação entre duas classes C_1 e C_2, então as duas assertivas a seguir são verdadeiras.

1. Cada objeto de C_1 conhece todos os objetos de C_2 aos quais ele está associado.
2. Cada objeto de C_2 conhece todos os objetos de C_1 aos quais ele está associado.

Graficamente, uma associação unidirecional é representada adicionando-se um sentido à associação. A classe para a qual o sentido aponta é aquela cujos objetos *não* possuem visibilidade dos objetos da outra classe. O diagrama da Figura 8-8 ilustra o uso de uma associação unidirecional. Esse diagrama indica que objetos da classe `Pedido` possuem, cada qual, uma referência para um objeto `Cliente`. Por outro lado, um objeto de `Cliente` não tem referências para os objetos associados em `Pedido`.

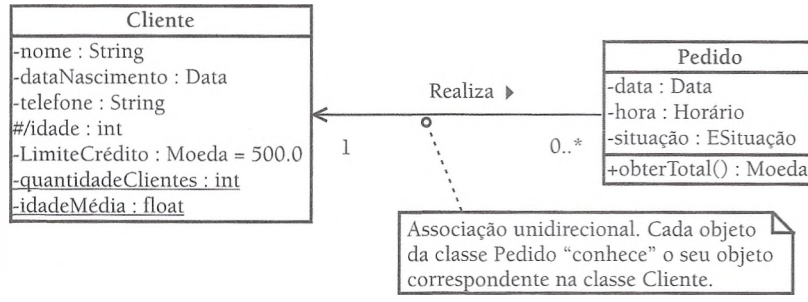


Figura 8-8: Exemplo de associação unidirecional.

Durante a construção do modelo de classes de análise, associações são costumeiramente consideradas "navegáveis" em ambos os sentidos, ou seja, as associações são bidirecionais. No modelo de classes de projeto, o modelador deve refinar a navegabilidade de todas as associações. Pode ser que algumas associações devam permanecer bidirecionais. No entanto, se não houver essa necessidade, recomenda-se transformar a associação em unidirecional. Isso porque uma associação bidirecional é mais difícil de implementar e de manter do que uma associação unidirecional correspondente. Para entender isso, basta ver que o *acoplamento* (ver Seção 7.5.2) entre as classes envolvidas na associação é maior quando a navegabilidade é bidirecional. Portanto, um dos objetivos a alcançar durante o projeto é identificar quais navegabilidades são realmente necessárias.

A definição do sentido da navegabilidade se dá em função dos diagramas de interação construídos para o sistema (ver Capítulo 7). Mais especificamente, devemos analisar os *fluxos das mensagens* entre objetos no diagrama de interações. Dados dois objetos associados, A e B, se há pelo menos uma mensagem de A para B em algum diagrama de interação, então a associação deve ser navegável da classe de A para a classe de B. Da mesma forma, se existir pelo menos uma mensagem de B para A, então a associação deve ser navegável de B para A.

A definição do sentido da navegabilidade é feita em função dos diagramas de interação construídos para o sistema. Devemos analisar os *fluxos das mensagens* entre objetos no diagrama de interações.

Mesmo em situações em que a navegabilidade de uma associação deva ser bidirecional, é possível implementar apenas um sentido da mesma. Por exemplo, considere a Figura 8-9. Observe, ainda, que é necessário saber de que corridas certo cavalo participou, e também os cavalos que participaram de certa corrida. Essas duas necessidades indicam uma associação bidirecional. Entretanto, se a coleção de cavalos não for tão grande, pode-se definir uma associação unidirecional no sentido de `Corrida` para `Cavalo`. Isso satisfaz a segunda necessidade. Para satisfazer a primeira necessidade, basta percorrer a coleção de corridas e, para cada uma, verificar se o cavalo em questão participou da mesma. Note que essa solução só se justifica se o tempo necessário para percorrer a coleção de corridas não tiver um grande impacto no desempenho. De um modo geral, se o desempenho for um fator crítico, é melhor implementar a associação nos dois sentidos.

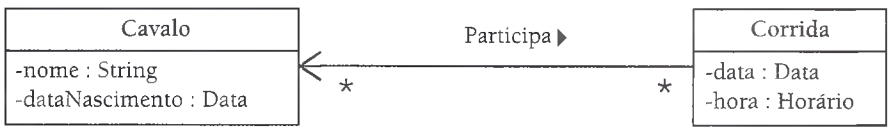


Figura 8-9: É possível substituir uma associação bidirecional por uma unidirecional.

8.4.4 Definindo a implementação de associações

Para associações de conectividade *um para um*, a implementação é trivial. Suponha que existam duas classes ligadas por uma associação de conectividade uma-para-um, C_a e C_b. Considere ainda que a navegabilidade definida é no sentido de C_a para C_b. Nesse caso, é definido um atributo do tipo C_b na classe C_a. Se a navegabilidade for bidirecional, o procedimento anterior pode ser aplicado para as duas classes. Portanto, em associações um para um, não há necessidade de um refinamento adicional.

Como um exemplo da forma típica utilizada para implementar associações de conectividade um-para-um, considere a Figura 8-10 e o Quadro 8-3. que apresenta um esboço de implementação (na sintaxe de linguagem Java) da

Figura 8-10: Exemplo de associação de conectividade um-para-um.

classe Professor. Note que há um atributo nessa classe cujo tipo é `GradeDisponibilidades`. Dessa forma, cada objeto da classe `Professor` possui uma referência para um objeto da classe `GradeDisponibilidades`, o que está consistente com o modelado na Figura 8-10.

Quadro 8-3: Implementação de associação de conectividade um-para-um

```
public class Professor {
   private GradeDisponibilidades grade;
   ...
}
```

Para o caso das associações de conectividade *um para muitos* ou *muitos para muitos*, o modelador pode decidir representar detalhes adicionais no modelo de classes de projeto, com o objetivo de esclarecer como tais associações devem ser implementadas. Uma razão para a representação desses detalhes adicionais pode ser o fato de que o sistema esteja sendo desenvolvido com o uso de uma ferramenta CASE (ver Seção 2.6) que gera código automaticamente a partir do diagrama de projeto. Nessa situação, o objetivo do modelador no detalhamento pode ser o de especificar a forma de implementação que a ferramenta deve utilizar. O detalhamento de associações de conectividade *um para muitos* ou *muitos para muitos* se baseia em classes que representam *coleções* de elementos. Vamos, então, descrever o conceito de *classe parametrizada*, que permite representar coleções em um diagrama de classes de especificação.

Uma coleção pode ser representada em um diagrama de classes por uma *classe parametrizada*. Uma classe parametrizada é uma classe utilizada para definir outras classes. Ela possui operações ou atributos cuja definição é feita em função de um ou mais parâmetros. Uma aplicação do conceito de classe parametrizada é na definição de classes que representam coleções. Independentemente do tipo de elementos que uma coleção possui, tal coleção deve possuir operações para adicionar um novo elemento, remover um elemento, encontrar um elemento etc. Sendo assim, uma coleção pode ser definida a partir de uma classe parametrizada, em que o parâmetro é o tipo do elemento da coleção.

Há duas notações possíveis na UML para representar uma classe parametrizada em um diagrama de classes. A Figura 8-11 é um exemplo das duas representações possíveis para a classe parametrizada denominada Coleção.

Figura 8-11: Notações possíveis na UML para uma classe parametrizada.

Uma questão interessante é acerca de qual é a correspondência do conceito de classe parametrizada em linguagens de programação. Na linguagem Java, por exemplo, classes parametrizadas são representadas pelo mecanismo de *tipos genéricos* (termo original: *Java Generics*). A linguagem C++ também possui uma forma de implementar classes parametrizadas, com o conceito de *templates*. A linguagem C# é outra que fornece o mecanismo de classes parametrizadas. Como um exemplo, considere o fragmento de classe apresentado no Quadro 8-4, no qual utilizamos a sintaxe da linguagem Java. Esse fragmento apresenta um atributo definido como um tipo genérico, `préRequisitos`. Nesse caso particular, esse atributo representa uma coleção de objeto cujos elementos são da classe `Disciplina`.

Quadro 8-4: Classe paramentrizada na sintaxe da linguagem Java

```
public class Disciplina {
   private Set<Disciplina> préRequisitos;
   ...
}
```

Conforme declaramos há pouco, o conceito de classe parametrizada pode ser utilizado para detalhar de que forma pode ser refinada uma associação de conectividade um-para-muitos ou muitos-para-muitos. Analisamos esse aspecto a seguir.

Em uma associação de conectividade "um-para-muitos", a forma de refiná-la (e, conseqüentemente, de implementá-la) depende da navegabilidade definida. Há dois casos a considerar. No primeiro caso, a navegabilidade aponta para o lado "muitos" da associação. No segundo caso, a navegabilidade aponta para o lado "um" da associação. Vamos agora analisar cada um desses casos.

Para refinar uma associação de conectividade um-para-muitos em que a navegabilidade é do lado "um" para o lado "muitos", a idéia básica é definir uma classe parametrizada cujo parâmetro é a classe correspondente ao lado "muitos" da associação. Como exemplo, a Figura 8-12 ilustra três versões de um mesmo fragmento de diagrama de classes para representar o fato de que um objeto da classe `Aluno` possui associação com diversos (muitos) objetos da classes `Participação`. A parte (a) ilustra a versão menos detalhada (apenas a navegabilidade é especificada). As partes (b) e (c) da Figura 8-12 ilustram duas representações mais detalhadas. Essas representações são equivalentes e utilizam o conceito de classe parametrizada. Conforme mencionamos na Seção 7.1.5, considera-se que a classe parametrizada correspondente à coleção possui operações que permitem adicionar, remover e obter acesso aos seus elementos. As linguagens de programação orientadas a objetos fornecem diversas classes que podem

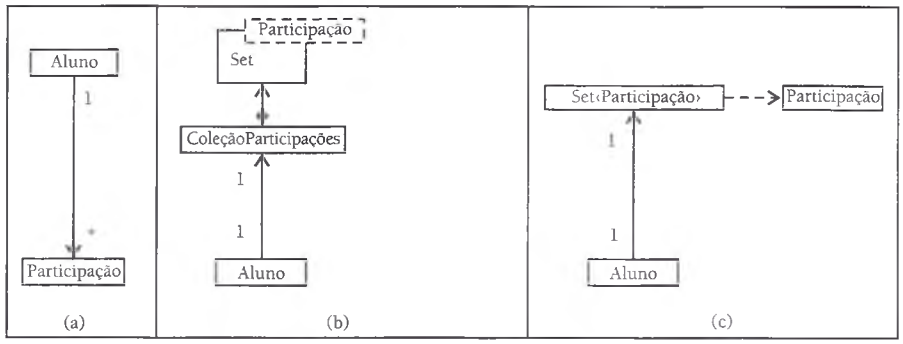


Figura 8-12: Formas alternativas para representar uma associação um para muitos.

ser utilizadas como a classe Coleção Contêiner. Note-se também que a utilização da classe parametrizada resultou na transformação de uma associação um para muitos em uma associação um para um.

Para dar uma idéia de como o diagrama da Figura 8-12 pode ser implementado, considere o Quadro 8-5. Esse quadro apresenta um trecho da implementação da classe `Aluno` em linguagem Java. Note a presença do atributo denominado "`participações`", uma lista parametrizada de objetos da classe `Participação`. Nesse exemplo, utilizamos a classe Set presente na linguagem Java. Também apresentamos os protótipos dos métodos `adicionarParticipação` e `removerParticipação`, que servem para manipular a coleção representada pelo atributo `participações` (ver Seção 8.3.6).

Quadro 8-5: Implementação de associação um-para-muitos e de navegabilidade para o lado "muitos"

```
public class Aluno {
   private Set<Participação> participações;
   ...

   public boolean adicionarParticipação(Participação p){
   ...
 }

   public boolean removerParticipação(Participação p) {
   ...
 }
}
```

O segundo caso possível de refinamento de uma associação um-para-muitos é aquele em que a mesma tem navegabilidade apontando para o lado "um". Nesse caso, a utilização de uma coleção (como no primeiro caso) não é

necessária. Quando a navegabilidade somente aponta para o lado "um", tudo o que deve ser feito é criar um atributo de referência na classe correspondente ao lado "muitos". O tipo desse atributo corresponde à classe que está no lado "um" da associação. Por exemplo, o Quadro 8-6 apresenta um exemplo de trecho de código resultante da aplicação dessa regra. Nesse quadro, temos a classe `OfertaDisciplina`, que possui uma associação com a classe `Disciplina`. A navegabilidade dessa associação é de `OfertaDisciplina` para `Disciplina` (ou seja, aponta para o lado "um"). Portanto, como mostra essa figura, criamos um atributo na classe `OfertaDisciplina` para referenciar o objeto `Disciplina` ao qual a primeira está associada.

Quadro 8-6: Implementação de associação muitos-para-muitos

```
public class OfertaDisciplina {
   private Disciplina disciplina;
   ...
}
```

A utilização do conceito de classe parametrizada no detalhamento de uma associação cuja conectividade é *muitos para muitos* é bastante semelhante ao caso das associações um para muitos. Considere o exemplo da Figura 8-13a, que ilustra uma associação entre as classes `Disciplina` e `GradeDisponibilidades`. Essa figura também informa que a associação é navegável no sentido de `GradeDisponibilidades` para `Disciplina`. A Figura 8-13b apresenta o detalhamento do diagrama apresentado na Figura 8-13a. Note que novamente uma classe parametrizada é utilizada no refinamento da associação.

Além do refinamento das associações comuns que descrevemos anteriormente, há também o caso das classes associativas (ver Seção 5.2.2.4). No refina-

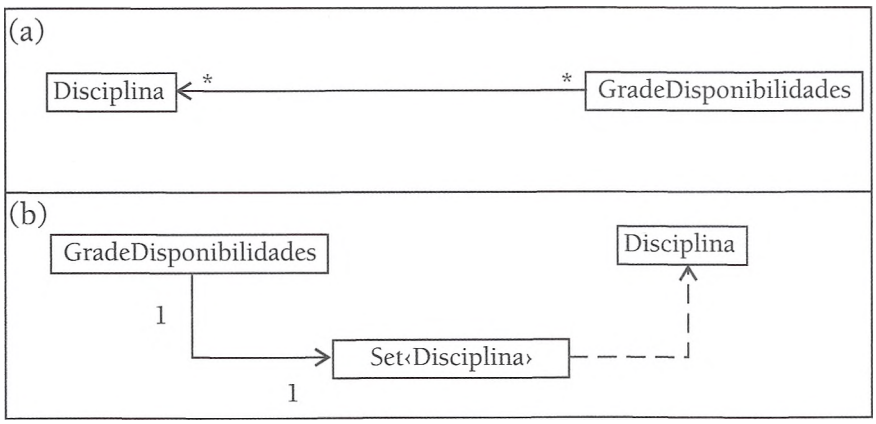


Figura 8-13: Associação muitos para muitos entre Disciplina e GradeDisponbilidades.

mento de classes associativas, a solução mais usual é criar uma classe para substituir a associativa. Uma vez feito isso, o problema se transforma no refinamento de associações um para muitos ou muitos para muitos. Como exemplo, a Figura 8-14 ilustra dois diagramas de classes. O diagrama da direita corresponde a um possível refinamento do diagrama da esquerda.

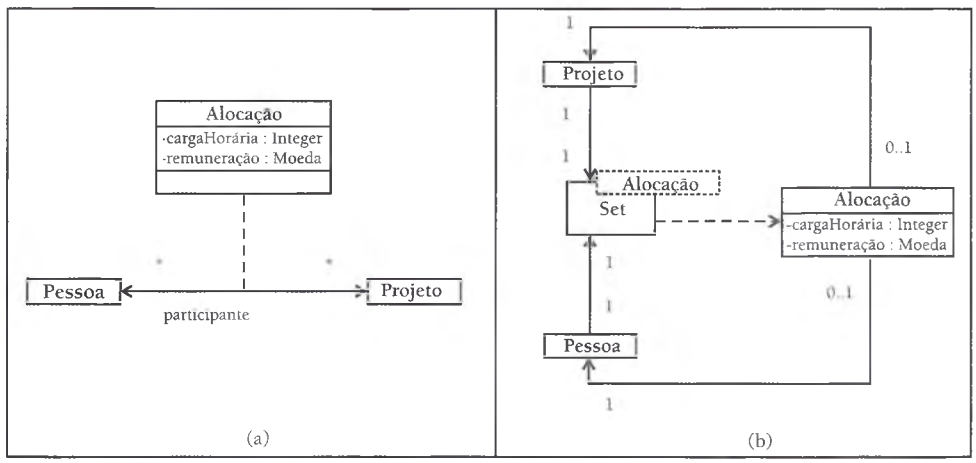


Figura 8-14: Refinamento de uma classe associativa.

8.5 Herança

Na Seção 5.2.3, estudamos o relacionamento de herança no contexto do modelo de classes de análise. Na modelagem de classes de projeto, há diversos aspectos relativos a esse relacionamento, que, nesta fase, normalmente é chamado de relacionamento de herança. Descrevemos esses aspectos nesta seção.

8.5.1 Tipos de herança

Há diversas categorizações que podem ser feitas a respeito de conceito de herança. Uma delas é com relação à quantidade de superclasses que certa classe pode ter. De fato, uma classe pode ter mais de uma superclasse. Essa situação corresponde ao conceito de *herança múltipla*. Esse conceito é ilustrado na Figura 8-15. Este diagrama representa um carro anfíbio, que possui tanto características de um carro quanto de um barco. Se uma classe tem apenas uma superclasse, então estamos diante da *herança simples*.

O uso de herança múltipla deve ser evitado. Um dos motivos para essa recomendação é que esse tipo de herança é difícil de entender. Além disso, algumas linguagens de programação não possuem construções para dar suporte à implementação da herança múltipla. Java, C# e Smalltalk, por exemplo, são lingua-

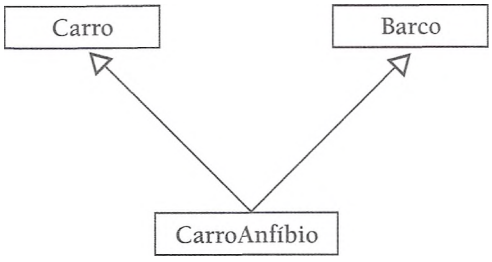


Figura 8-15: Exemplo de herança múltipla.

gens orientadas a objetos que não dão suporte direto ao conceito de herança múltipla. Por fim, a maioria das linguagens de programação mais modernas possui uma alternativa para o uso de herança múltipla, as *interfaces*, que descrevemos na Seção 8.5.4.

Outro tipo de categorização do conceito de herança é com relação à forma de reutilização envolvida. Há duas formas de reutilização por herança em LPOO modernas (Java, C# etc.): *herança de implementação* e *herança de interface*. Para estabelecer a diferença entre esses dois tipos, devemos primeiramente definir o conceito de *serviço*. Um serviço é composto de sua *especificação* e de seu *método*. A *especificação* define *o que* o serviço realiza, além de documentar as informações necessárias para que o serviço seja realizado e o resultado da sua realização. Já o *método* corresponde ao modo de realizar um serviço. Dada uma especificação de um serviço, podem existir diversos métodos para realizá-lo. Na herança de implementação, uma subclasse herda métodos de um "ancestral". Já na herança de interface, uma subclasse herda as especificações definidas na interface de um "ancestral" e se compromete a implementar essa interface. A herança de implementação é bastante intuitiva; é fácil entender que uma subclasse herda os métodos de qualquer operação definida em seu ancestral. Entretanto, o conceito de herança de interface não é tão intuitivo. Para esclarecer esse tipo de herança, nas próximas seções, introduzimos os conceitos de *classe abstrata* e de *interface*.

8.5.2 Classes abstratas

É comum, a existência de uma classe se justificar pelo fato de haver a possibilidade de gerar instâncias da mesma. Classes que geram instâncias são chamadas de *concretas*. No entanto, podem existir classes que não geram instâncias (objetos) diretamente. Essas classes são normalmente utilizadas para organizar e simplificar uma hierarquia de herança. Portanto, classes abstratas só existem em hierarquias. Propriedades comuns a diversas classes podem ser organizadas e definidas em uma única classe abstrata da qual as primeiras herdam.

Na notação da UML, uma classe abstrata é representada com o seu nome em *itálico*. A Figura 8-16 apresenta um exemplo de classe abstrata que serve como

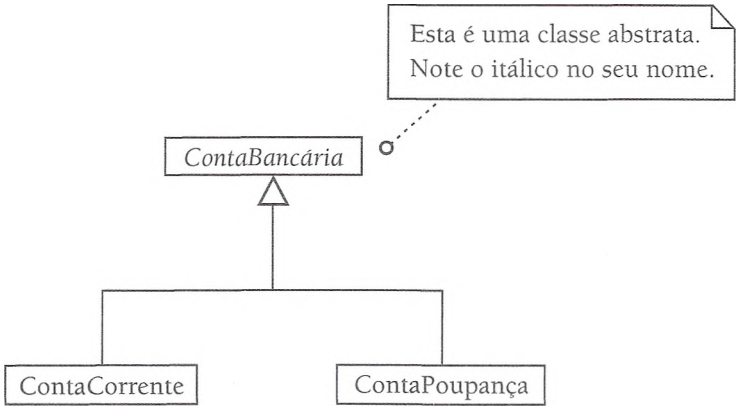


Figura 8-16: Exemplo de classe abstrata.

superclasse para duas outras classes. Uma notação alternativa é utilizar a etiqueta {abstract} (ver Seção 3.3) dentro do compartimento do nome da classe.

Subclasses de uma classe abstrata também podem ser abstratas, mas a hierarquia deve terminar em uma ou mais classes concretas. Ou seja, não faz sentido existir uma hierarquia de herança com uma classe abstrata na base dessa hierarquia.

Além de não poder ser diretamente instanciada, outra característica associada ao conceito de classe abstrata é o fato de ela possuir ao menos uma *operação abstrata*. Na terminologia da Seção 8.5.1, uma operação abstrata corresponde à *especificação* de um serviço que a classe deve fornecer (sem *método*). Uma classe qualquer pode possuir tanto operações abstratas, quanto operações concretas (ou seja, operações que possuem implementação). Entretanto, uma classe que possui pelo menos uma operação abstrata é, por definição, abstrata. Assim como acontece com qualquer operação pública, uma operação abstrata definida em uma classe também é herdada por suas subclasses. Quando uma subclasse herda uma operação abstrata e não fornece uma implementação para a mesma, esta classe também será abstrata (pois, por herança, passará a possuir uma operação abstrata). Por outro lado, se esta classe fornecer uma implementação para quaisquer operações abstratas herdadas, e ela própria não define operações abstratas, esta classe é concreta.

Graficamente, uma operação abstrata é representada em *itálico* no diagrama de classes, segundo a notação definida pela UML. O uso da etiqueta {abstract} também é uma alternativa. Como exemplo, considere a Figura 8-17. Nesta figura, a classe `FiguraGeométrica` é abstrata, pois possui uma operação abstrata, *desenhar* (note que a assinatura dessa operação está declarada em itálico). As classes `Círculo` e `Quadrado` são concretas, pois fornecem implementação para a operação abstrata herdada.

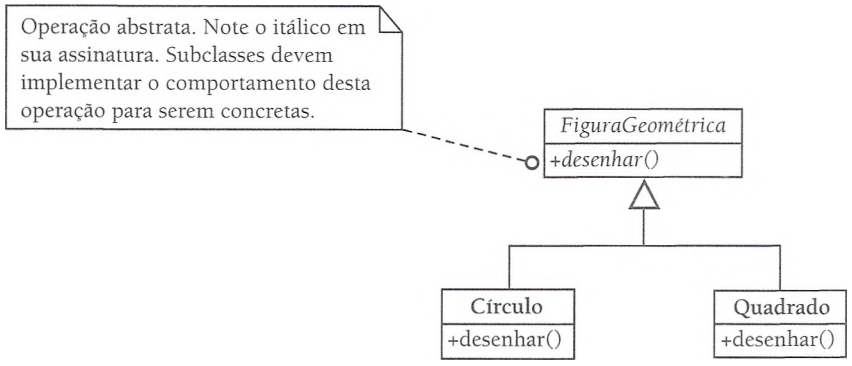


Figura 8-17: Herança de operação abstrata.

8.5.3 Operações polimórficas

Uma subclasse herda todas as propriedades de sua superclasse que tenham visibilidade pública ou protegida. Isso quer dizer que é possível a situação em que um objeto da subclasse recebe uma mensagem para que uma operação pública ou protegida definida em sua superclasse seja executada. Entretanto, pode ser que o comportamento de alguma operação herdada tenha que ser diferente para a subclasse. Nesse caso, a subclasse deve redefinir o comportamento da operação. Note que a assinatura da operação é reutilizada pela subclasse; tudo o que a subclasse faz é implementar o novo comportamento desejado para a operação. O resultado disso é que a subclasse e a superclasse agora compartilham a mesma assinatura de certa operação, mas cada uma delas possui um *método* (forma de implementação) particular para essa operação.

Por definição, *operações polimórficas* são operações de mesma assinatura definidas em diversos níveis de uma hierarquia de herança e que possuem métodos diferentes. Essas operações devem ter a assinatura repetida na(s) subclasse(s) para enfatizar que elas estão sendo redefinidas (somente a assinatura é herdada, mas não a implementação).

Como exemplo, considere as classes `Funcionário` e `Vendedor`, esta última subclasse da primeira. Em `Funcionário`, existe um método `obterPagamento`, que retorna o valor do pagamento do funcionário. Suponha que o método para obter o valor do pagamento de um vendedor *não* seja o mesmo para obter o pagamento de um funcionário. Nesse caso, não faz sentido `Vendedor` herdar a implementação dessa operação. Em vez disso, `Vendedor` deve *redefinir* a operação `obterPagamento`. Diz-se, então, que essa operação é polimórfica. A Figura 8-18 ilustra graficamente essse exemplo. A assinatura da operação `obterPagamento` é repetida em `Vendedor`. (As notas explicativas ([ver Seção 3.2]) apresentam as expressões de cálculo do pagamento utilizadas em uma classe e na outra.) Note que, como as operações têm a mesma assinatura, a

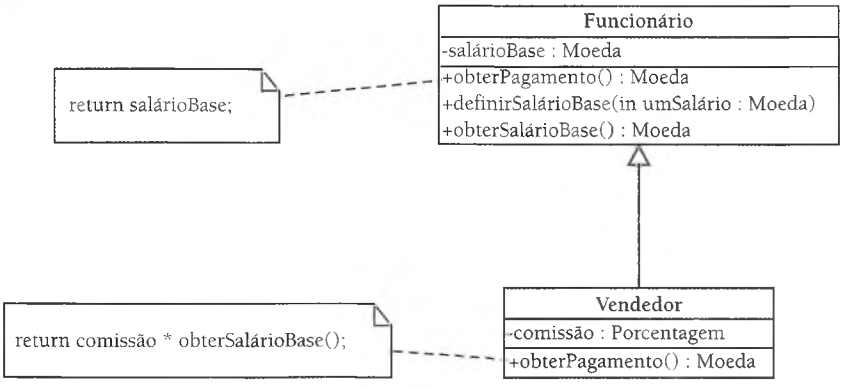


Figura 8-18: Definição de operações polimórficas.

mensagem utilizada para obter o valor do pagamento de um funcionário ou de um vendedor é a mesma.

Operações polimórficas também podem existir em classes abstratas. A Figura 8-19 ilustra um exemplo dessa situação. Nesse exemplo, há uma classe abstrata denominada `ContaBancária`. Essa classe possui duas subclasses, `ContaCorrente` e `ContaBancária`. Note também que a operação aplicarJuros em `ContaBancária` também é abstrata (pois sua assinatura está declarada em itálico). Isso significa que as subclasses devem fornecer uma implementação a essa operação para que também não se tornem abstratas. Portanto, tanto `ContaCorrente` quanto `ContaPoupança` implementam a operação herdada.

Operações polimórficas são importantes porque ajudam na implementação do *princípio do polimorfismo* no qual objetos que pertencem a classes diferentes respondem à mesma mensagem. (O princípio do polimorfismo é apresentado na Seção 1.2.3.2.) A utilização de operações polimórficas tem o

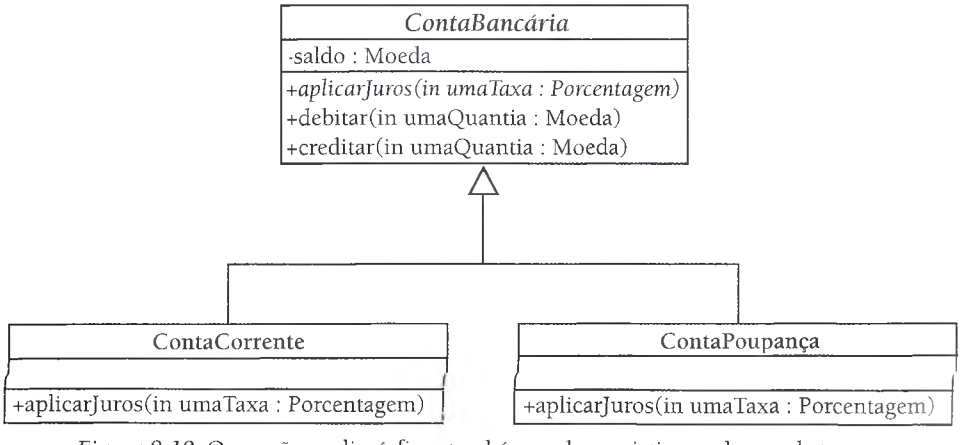


Figura 8-19: Operações polimórficas também podem existir em classes abstratas.

objetivo de garantir que as subclasses compartilhem uma mesma operação, mas com métodos diferentes. Em outras palavras, se duas ou mais subclasses de uma mesma classe implementam a mesma operação polimórfica, o remetente da mensagem não precisa saber qual a verdadeira classe de cada objeto receptor, pois eles aceitam a mesma mensagem. A diferença é que o método que implementa a operação a ser executada é diferente em cada subclasse. Como exemplo disso, considere o trecho de código apresentado no Quadro 8-7. Nesse exemplo, há uma coleção cuja definição indica que ela armazena objetos da classe `ContaBancária`. A princípio, esse exemplo parece incoerente com a definição de classe abstrata: como pode existir uma coleção cujos elementos são objetos de uma classe da qual não pode haver instâncias? A resposta para a aparente incoência é que, embora não possa haver instâncias diretas da classe `ContaBancária`, pode haver instâncias *indiretas* da mesma. Note que instâncias das classes `ContaCorrente` e `ContaPoupanca` são indiretamente instâncias da classe `ContaBancária`. Portanto, a coleção denominada contasBancárias pode conter objetos das subclasses de `ContaBancária`. Agora note a região de código destacada (em negrito) no Quadro 8-7. Essa região corresponde a uma iteração que percorre todos os objetos na coleção contasBancárias e, para cada um deles, invoca a execução da operação aplicarJuros. Como essa coleção contém objetos de ambas as subclasses de `ContaBancária`, os métodos invocados serão diferentes, em função da verdadeira classe do objeto receptor da mensagem. De qualquer modo, o fato importante nesse exemplo é que há uma região de código (a que está destacada no Quadro 8-7) que envia a mensagem para objetos de classes diferentes, *sem saber qual é a verdadeira classe dos objetos receptores* (`ContaCorrente` ou `ContaPoupanca`). Essa é a essência do princípio do polimorfismo.

Quadro 8-7: Operações polimórficas ajudam na implementação do *polimorfismo*

```
ContaCorrente cc;
ContaPoupanca cp;
...
List<ContaBancária> contasBancárias;
...
contasBancárias.add(cc);
contasBancárias.add(cp);
...
for(ContaBancária conta:  contasBancárias) {
   conta.aplicarJuros(0.05);
}
...
```

8.5.4 Interfaces

De acordo com a definição de um dicionário, uma interface é um dispositivo ou um sistema que entidades não relacionadas utilizam para se comunicar. No contexto do desenvolvimento de sistemas orientados a objetos, quando estamos construindo uma comunidade de entidades (objetos) que interagem entre si, uma interface representa a "cara" que uma dessas entidades mostra à outra, ou seja, que serviços ela fornece. Uma entidade pode, naturalmente, ter muitas interfaces, mostrando as "caras diferentes" a diferentes membros de comunidade de entidades.

Considere o exemplo a seguir para entender o conceito de interface. Suponha que exista uma classe `Bicicleta`, que pertence a uma hierarquia de classes (veículos). A classe `Bicicleta` define o que uma bicicleta pode fazer em termos do comportamento de um veículo. No entanto, um objeto bicicleta pode interagir com o mundo de outras formas. Por exemplo, uma bicicleta poderia ser um dos produtos manipulados por um sistema de vendas. Esse sistema provavelmente não necessita do comportamento relativo a um veículo da classe `Bicicleta`, apenas que objetos dessa classe se comportem como produtos que possam ser vendidos e que forneçam certas informações (por exemplo, preço de venda, número de registro etc.). Ou seja, para ser manipulado pelo sistema de vendas, um objeto da classe bicicleta deve estar em concordância com o *protocolo ou contrato de comportamento* predefinido. Este protocolo compreende um conjunto não vazio de *assinaturas de operações*. Cada assinatura apresenta o nome da operação, juntamente com a definição de seus eventuais parâmetros e tipo de retorno. O protocolo de interação não contém a implementação das operações nele definidas. Em uma linguagem de programação orientada a objetos, esses protocolos são denominados *interfaces*.

Voltando ao exemplo do sistema de vendas, este pode interagir com os objetos a serem vendidos (produtos) por intermédio de uma interface; não há necessidade desse sistema conhecer as verdadeiras classes dos objetos com os quais interage. Dessa forma, objetos da classe `Bicicleta` ou da classe `Geladeira` podem ser tratados de forma indistinta pelo sistema de vendas. De uma forma geral, qualquer classe que implemente (forneça implementação) para as operações definidas na interface que o sistema de vendas espera pode representar o papel de produto. Além disso, se por um lado uma interface é usada para definir um *contrato de comportamento*, por outro lado o mesmo contrato de comportamento pode ser implementado por diferentes classes. A conseqüência disso é que não há a necessidade de definirmos relacionamentos diretos entre classes que, em situações normais, não estariam relacionadas. Em vez disso, podemos definir uma interface para que essas classes se comuniquem. Podemos então definir os seguintes objetivos do conceito de interface:

1. Capturar semelhanças entre classes não relacionadas sem forçar relacionamentos entre elas.

2. Declarar operações que uma ou mais classes devem implementar.
3. Revelar as operações de um objeto, sem revelar a sua classe.
4. Facilitar o desacoplamento entre elementos de um sistema.

Uma interface corresponde a um conjunto de especificações de *serviços* (ver Seção 8.5.1) fornecidos por um *classificador*. Um classificador é conceito da UML usado para denotar uma *classe*, um *subsistema* ou um *componente*. (Estes dois últimos elementos são descritos nas Seções 11.1 e 11.2.2, respectivamente.) Um classificador possui comportamento, ou seja, implementa um conjunto de operações. Através dessas operações, um classificador pode fornecer serviços para outros classificadores. Ele também pode utilizar serviços destes últimos. Um classificador pode implementar várias interfaces, assim como uma interface pode ser implementada por vários classificadores. Em uma interface, não há métodos associados às especificações. O classificar é que fornece métodos para as especificações das classes que ele implementa.

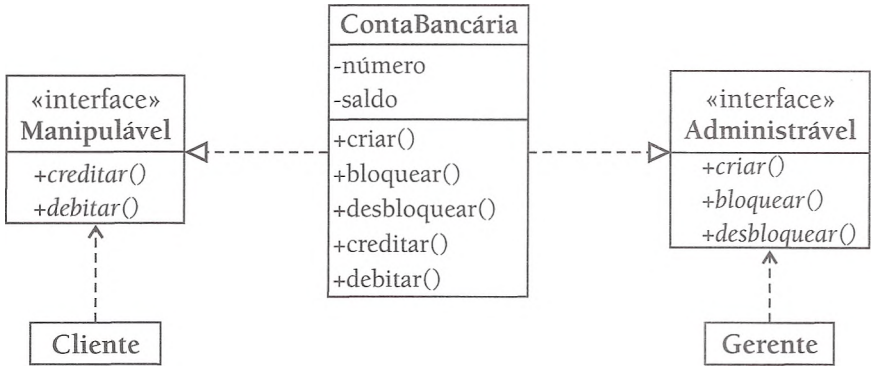


Figura 8-20: A classe ContaBancária implementa duas interfaces: Manipulável e Administrável.

A UML define duas notações equivalentes para representar graficamente uma interface. (Os exemplos a seguir são fornecidos em relação a classes, mas a notação vale para qualquer classificador.) A primeira notação para uma interface é a mesma definida para classes, em que são exibidas as operações que a interface define. Entretanto, o compartimento dos atributos fica sempre vazio, pois uma interface não possui atributos. Além disso, no compartimento do nome da interface deve aparecer o estereótipo `<<interface>>`. Como exemplo, a Figura 8-20 utiliza a primeira notação e indica que `ContaBancária` realiza as interfaces `Manipulável` e Administrável. A primeira interface (`Manipulável`) especifica as operações `creditar( )` e `debitar( )`. A segunda interface (`Administrável`) especifica as operações `criar( )`, `bloquear( )` e `desbloquear( )`. Para as classes `Cliente` e `Gerente`, que utilizam as interfaces que `ContaBancária` realiza, tudo se passa como se estivessem utilizando a própria classe. A diferença é que essas classes somente visualizam as operações que fazem sentido para elas. Além disso, qualquer outra classe que im-

plemente as interfaces `Manipulável` e `Administrável` pode substituir a classe `ContaBancária` sem modificações nas demais classes. A segunda notação possível para uma interface na UML é através de um segmento de reta com um pequeno círculo em um dos extremos e ligado à classe que a realiza (implementa) no outro extremo. O nome da interface é posicionado próximo ao extremo do segmento que contém o pequeno círculo. Nesta notação mais simplificada, as operações da interface não são ilustradas. As classes clientes estão conectadas à interface através de um relacionamento de dependência. A Figura 8-21 exibe essa segunda forma de representação de interfaces. Note que, em ambas as formas de representação de interfaces, a linha de dependência parte de um classificador e termina no símbolo de uma das interfaces de outro classificador.

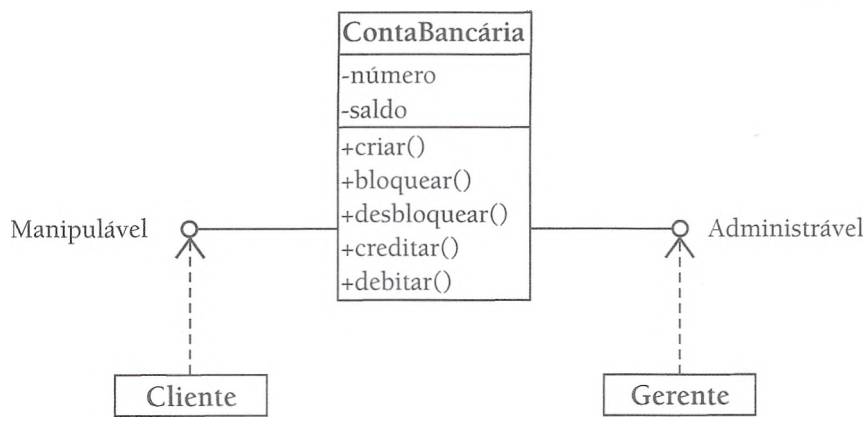


Figura 8-21: Exemplo da forma simplificada para representar interfaces.

Conforme mencionado há pouco, uma classe pode realizar (implementar) várias interfaces. Nesse caso, a classe deve fornecer implementações (métodos) para as operações cujas assinaturas (especificações) são declaradas nas interfaces. Por exemplo, a classe `ContaBancária` fornece implementações para as operações declaradas nas interfaces `Manipulável` e `Administrável`. Também pode ser que a mesma interface seja realizada por várias classes. Nesse caso, essas classes devem implementar as operações da interface que realizam. Por exemplo, se uma interface I é realizada por três classificadores A, B e C, e declara as operações `op1( )`, `op2( )`, `op3( )` e `op4( )`, então essas operações devem estar implementadas em alguns dos classificadores.

Algumas linguagens de programação possuem em sua sintaxe suporte direto ao conceito de interface, mas outras não. As linguagens Java e C# apresentam o conceito de interface em suas sintaxes. A linguagem C++, por outro lado, não possui explicitamente o conceito de interface. Mesmo assim, nessa linguagem, interfaces podem ser definidas indiretamente por classes abstratas em que todas as operações são abstratas e não há atributos.

Juntamente com o conceito de classes abstratas, interfaces permitem alcançar o *acoplamento abstrato*. Descrevemos esse conceito na próxima Seção.

8.5.5 Acoplamentos concreto e abstrato

Na Seção 7.5.2, descrevemos o conceito de *acoplamento*. Vimos que esse conceito corresponde a uma medida da *dependência* existente entre classes de um sistema. Vimos também que devemos manter o acoplamento em um nível mínimo possível, com o objetivo de diminuir as dependências na estrutura de classes correspondente. Note a utilização do termo "mínimo possível" na frase anterior. O acoplamento é necessário quando um objeto precisa solicitar serviços (interagir) de outro com o envio de mensagens. Nesse caso, o objeto remetente precisa ter uma *referência* para o receptor.

A forma usual de um objeto ter uma referência para outro é fazer com que o remetente *tenha conhecimento direto da classe do receptor*. O tipo de dependência corresponde ao chamado *acoplamento concreto*. Esse nome surge do fato de que há duas *classes concretas* (ver Seção 8.5.2) envolvidas. Entretanto, há outra forma de dependência que permite que um objeto remetente envie uma mensagem para um receptor *sem ter conhecimento da verdadeira classe desse último*; essa forma de dependência corresponde ao *acoplamento abstrato*. Mas como o acoplamento abstrato pode ser possível? Para entendermos isso, vamos analisar novamente o conceito de *interface*, que descrevemos na Seção 8.5.4. Acompanhe essa análise a seguir, juntamente com a Figura 8-22, que ilustra esquematicamente os acoplamentos concreto e abstrato.

Uma interface pode ser interpretada como um nível de *indireção*, pelo qual classificadores podem utilizar os serviços uns dos outros. Para entender isso, considere dois classificadores, Ca e Cb1. Suponha que Ca utiliza serviços de Cb1 através de uma interface ICb. Se um novo classificador Cb2 que implementa a mesma interface ICb foi desenvolvido, este pode substituir Cb1 sem que Ca precise sofrer modificações quando essa substituição ocorrer. Para entender isso, note que Ca utiliza os serviços implementados em Cb1 por intermédio da interface ICb. Ou seja, Ca não tem uma referência direta para uma instância de Cb. Com efeito, o uso de interfaces entre classificadores torna o sistema mais flexível a mudanças. Os classificadores fornecedores de algum serviço podem ser substituídos sem causar modificações nos classificadores clientes, desde que a interação ocorra através de uma interface. Essa é a essência do acoplamento abstrato. Interfaces permitem encapsular comportamento, ocultando qual a classe de um objeto que está realizando uma tarefa específica. Isso é possível porque a classe que implementa uma interface é obrigada a seguir o protocolo definido nesta última.

Conforme também podemos inferir da Figura 8-22, o acoplamento abstrato pode ser alcançado não só com o uso de interfaces, mas também com *classes abs-*

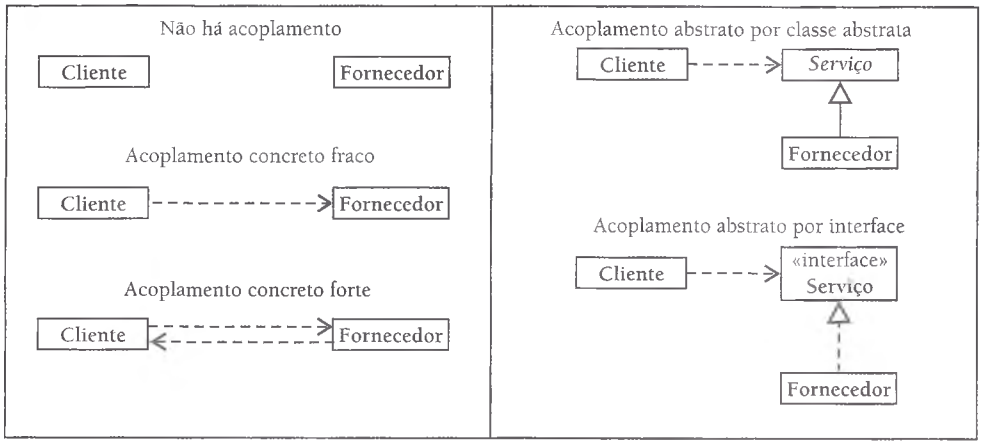


Figura 8-22: Formas de acoplamento entre classificadores.

tratas (ver Seção 8.5.2). A razão disso é que o conceito de interface guarda diversas semelhanças com o conceito de classe abstrata, conforme descrevemos a seguir. Em primeiro lugar, da mesma forma que uma classe abstrata, uma interface não gera instâncias. Além disso, tanto interfaces quantos classes abstratas possuem um conjunto de operações abstratas. Note, entretanto, que há diferenças entre esses dois conceitos. Ao contrário de uma classe abstrata, uma interface não pode conter estrutura interna alguma (ou seja, não pode ter atributos, nem associações). Além disso, todas as operações de uma interface só possuem especificações; a implementação (método) de cada operação não é definida na interface.

Uma interface (ou uma classe abstrata) descreve uma parte do comportamento externamente visível de um conjunto de objetos, possivelmente de classes diferentes. O acoplamento abstrato entre elementos (classificadores) que podemos alcançar com o uso de uma interface ou de uma classe abstrata é muito importante na construção de sistemas orientados a objetos de qualidade. Quando é bem utilizado, esse tipo de acoplamento aumenta a flexibilidade e a capacidade de reutilização do sistema, assim como a qualidade do código-fonte. Isso porque o acoplamento abstrato nos permite isolar um conjunto de serviços de certa aplicação que são instáveis, no sentido de terem alto potencial de precisarem de modificações em sua implementação no futuro. Se detectamos algum conjunto de serviços com essa característica de instabilidade, podemos isolá-lo por trás de uma interface (ou classe abstrata). Os clientes desse conjunto de serviços têm acesso a eles por intermédio da interface, sem precisarem ter conhecimento direto das classes que os implementam. Se a implementação de algum serviço precisar ser alterada, não precisaremos modificar os clientes do serviço. Nos padrões de projeto que descrevemos na Seção 8.6, o acoplamento abstrato é um princípio fundamental.

8.5.6 Reuso através de delegação

Na Seção 8.5.1, descrevemos duas formas de reuso por herança: *reuso de interface* e *reuso de comportamento*. Este último se baseia na noção de que subclasses herdam *comportamento* de sua superclasse. Por exemplo, na Figura 8-23, quando um objeto da classe `ContaCorrente` recebe uma mensagem para executar a operação debitar, ele não tem como atender a essa mensagem só com os recursos de sua classe. Ele, então, utiliza a operação herdada da superclasse. A herança de comportamento é um mecanismo fácil de implementar; tudo que o desenvolvedor deve fazer na definição de uma subclasse é adicionar novas propriedades às existentes na superclasse, ou redefinir as propriedades herdadas para se adequarem ao seu comportamento. No entanto, a herança por comportamento tem a desvantagem potencial de expor as subclasses aos detalhes de sua superclasse, violando assim o *princípio do encapsulamento* (ver Seção 1.2.3.1).

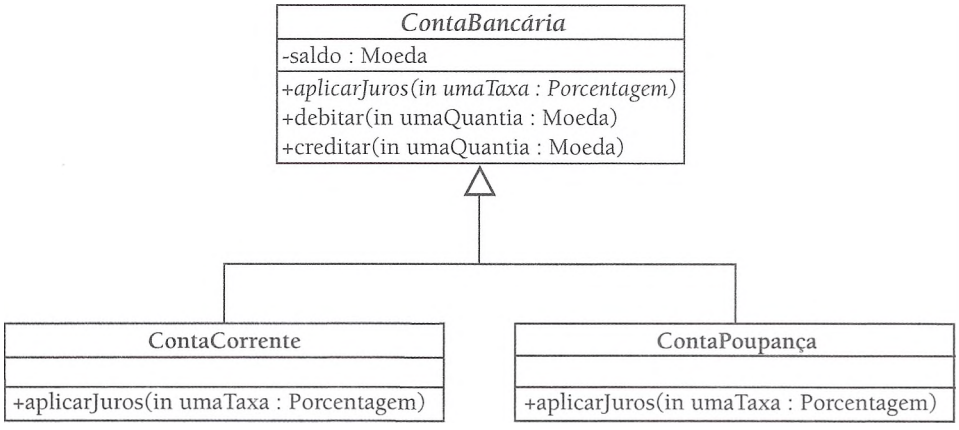


Figura 8-23: Redefinição de operações abstratas herdadas da superclasse.

O reuso de comportamento pode se dar não só pela herança entre classes, mas também pelo mecanismo de *delegação*. No reuso por delegação, sempre que um objeto não pode realizar uma tarefa (ou parte dela) por si próprio, ele delega a realização da mesma a outro(s) objeto(s). Nesse sentido, podemos afirmar que o objeto *reusa* as operações dos objetos para os quais ele delega responsabilidades.

Como um exemplo que serve de comparativo para as duas estratégias de reuso de comportamento, a Figura 8-24 ilustra a definição de uma classe `Pilha` segundo duas alternativas. Na primeira alternativa, a definição é feita pelo reuso por herança: a classe `Pilha` é definida com uma subclasse de `Vetor`. Note que a classe `Pilha` herda também operações que não fazem sentido para objetos dessa classe (por exemplo, elementos não podem ser inseridos em qualquer posição de uma pilha, assim como ocorre em um vetor). Na segunda alternativa, a defi-

nição é feita pela delegação. Um objeto `Pilha` agrega (tem como componente) um objeto `Vetor`. As operações na classe `Pilha` são implementadas pela delegação de mensagens para o objeto `Vetor`. Dessa forma, os clientes da classe `Pilha` somente têm acesso às operações definidas nessa classe. Isso porque o objeto Vetor está encapsulado na implementação da classe `Pilha`.

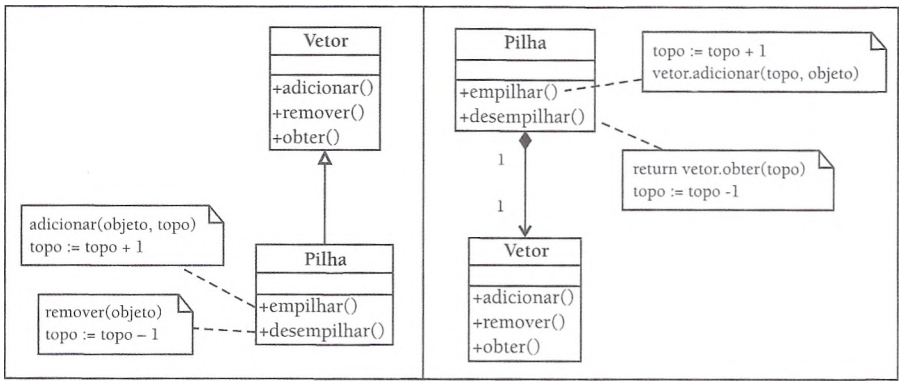


Figura 8-24: Duas definições da classe Pilha reutilizando a classe Vetor.

A delegação é mais genérica que a herança entre classes, pois um objeto pode reutilizar o comportamento de outro sem que o primeiro precise ser uma subclasse do segundo. Outra vantagem da delegação sobre a herança é que o compartilhamento de comportamento e o reuso podem ser realizados em tempo de execução. Na herança de classes, o reuso é especificado estaticamente. No entanto, a delegação pode apresentar perda de desempenho quando comparada a uma solução que use herança. Isso porque a delegação implica cruzar a fronteira de um objeto a outro para enviar a mensagem de delegação. Concluindo, há vantagens e desvantagens tanto no reuso por heranças e por delegação, e a utilização de uma ou outra depende da consideração de diversos fatores. De uma forma geral, *não* se recomenda utilizar herança nas seguintes situações:

- Para representar papéis de uma superclasse.
- Quando a subclasse for herdar propriedades que não se aplicam a ela.
- Quando um objeto de uma subclasse pode se transformar em um objeto de outra subclasse. Por exemplo, um objeto da classe Cliente se transforma em um objeto da classe Funcionário.

8.5.7 Classificação dinâmica

Na maioria dos casos práticos em que a herança deve ser utilizada, temos o caso da *classificação estática*. Segundo essa classificação, desde o momento em que um objeto é instanciado até o momento em que ele é destruído, sua classe per-

manece a mesma. Entretanto, um problema com o qual o modelador pode se deparar na especificação e implementação de uma relacionamento de herança é a *classificação dinâmica*, também chamada de *metamorfose*. Para entender esse conceito, considere uma empresa em que há empregados e clientes. Pode ser que uma pessoa, em um determinado momento, seja apenas cliente; depois pode ser que ela passe a ser também um empregado da empresa. A seguir, essa pessoa é desligada da empresa, continuando a ser cliente. Ou seja, um mesmo objeto pode pertencer a diferentes classes durante a sua vida (daí o nome *metamorfose*). Mais que isso, um mesmo objeto pode pertencer a múltiplas classes simultaneamente (por exemplo, uma pessoa que é, ao mesmo tempo, cliente e funcionário da empresa).

As principais linguagens de programação orientadas a objetos (C++, Java, as linguagens .NET, Smalltalk) não dão suporte direto à implementação da classificação dinâmica. Nessas linguagens, se um objeto é instanciado como sendo de uma classe, ele não pode pertencer posteriormente a uma outra classe.

Uma solução parcial para o problema da classificação dinâmica é definir todas as possíveis subclasses em uma determinada situação. Utilizando essa solução, o problema da empresa descrito há pouco seria modelado como uma hierarquia de classes em que haveria as subclasses `Empregado` (objetos que são só empregados), `Cliente` (objetos que são só clientes) e `Empregado-Cliente` (objetos que são tanto clientes quanto empregados). Essa solução não resolve o problema todo, pois pode ser que um objeto mude de classe. Além disso, considere que o conceito de gerente tenha de ser adicionado à hierarquia de herança. Isso elevaria o número de subclasses de três para cinco (fica como exercício para o leitor verificar esse fato), tornando o modelo ainda mais complexo.

Uma melhor solução para o problema da classificação dinâmica é utilizar a técnica de delegação descrita na Seção 8.5.6. Por meio dessa técnica, o relacionamento de herança existente entre cada subclasse e a superclasse é substituído

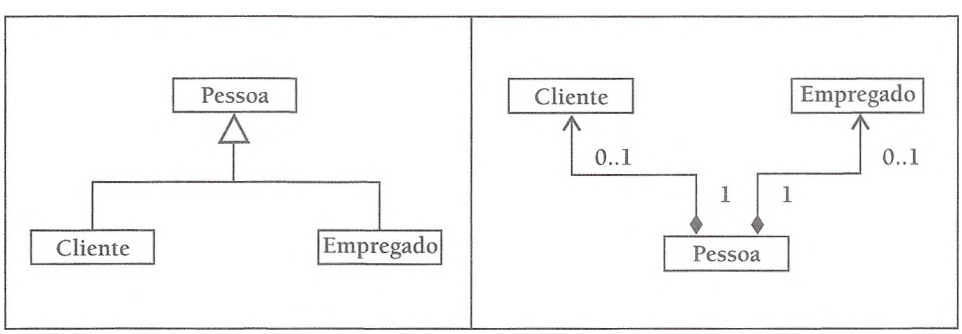


Figura 8-25: Exemplo de solução para o problema da classificação dinâmica.

por uma composição. Como um exemplo dessa solução, considere a Figura 8-25. A parte esquerda da figura exibe um fragmento de diagrama de classes de análise que apresenta uma hierarquia de classes. A parte direita da figura apresenta uma possível reestruturação feita na especificação para solucionar o problema da classificação dinâmica. Note que, no diagrama reestruturado, uma pessoa pode apresentar comportamento somente de um cliente, apenas de um empregado, ou de ambos.

Como um comentário final sobre a classificação dinâmica, note que durante a construção do modelo de análise, o modelador não deve se preocupar com esse problema. Portanto, nada impede que haja uma hierarquia de classes em um modelo de análise na qual um objeto possa pertencer a mais de uma subclasse, ou possa mudar de subclasse. Entretanto, quando chega a fase de projeto, o modelador deve considerar a reestruturação da hierarquia de classes para resolver o problema da classificação dinâmica.

8.6 Padrões de projeto

É da natureza do desenvolvimento de software o fato de que os mesmos problemas tendem a acontecer diversas vezes. Na Seção 5.4.4, descrevemos alguns padrões de análise, ou seja, padrões de software úteis para serem utilizados em problemas de modelagem recorrentes na fase de análise do desenvolvimento de um SSOO. Outra categoria de padrões de software extremamente útil são os padrões de projeto (tradução para *design patterns*). O texto clássico sobre esse assunto é o livro de Eric Gamma e seus colaboradores (Gamma et *al.*, 2000). Esses autores são popularmente conhecidos como equipe GoF (de *Gang of Four*, por conta de serem quatro autores). Nesse livro, os autores catalogaram 23 padrões de projeto. Os padrões de projeto descritos pela equipe GoF foram divididos em três categorias, descritas a seguir.

1. *Criacionais*: procuram separar a operação de uma aplicação de como os seus objetos são criados.
2. *Estruturais*: provêem generalidade para que a estrutura da solução possa ser estendida no futuro.
3. *Comportamentais*: utilizam herança para distribuir o comportamento entre subclasses, ou agregação e composição para construir comportamento complexo a partir de componentes mais simples.

Os 23 padrões documentados pela equipe GoF são enumerados na Tabela 8-5. Uma descrição detalhada de todos os padrões enumerados mais adiante está fora do escopo deste livro. Para um estudo mais detalhado sobre o assunto, o leitor pode consultar a referência (Gamma, 1995). Outra referência importan-

te é a que descreve os padrões de projeto do catálogo J2EE (Alur *et al.*, 2003). Por outro lado, consideramos importante, mesmo em um livro introdutório como este, dar uma visão geral de um assunto tão importante para o desenvolvimento de um SSOO. Para isso, descrevemos nas próximas seções três dos padrões de projeto representativos da categoria GoF. São eles: Composite, Observer, Strategy, Factory Method, Mediator e Façade.

Tabela 8-5: Padrões de projeto documentados pelo catálogo GoF

Criacionais	Estruturais	Comportamentais
Abstract Factory	Adapter	Chain of Responsibility
Builder	Bridge	Command
Factory Method	Composite	Interpreter
Prototype	Decorator	Iterator
Singleton	Façade	Mediator
	Flyweight	Memento
	Proxy	Observer
		State
		Strategy
		Template Method
		Visitor

Um padrão de projeto corresponde a um esboço de uma solução reusável para um problema comumente encontrado em um contexto particular. Estudar padrões é uma maneira eficaz de aprender com a experiência de outros.

8.6.1 Composite

O padrão Composite é um dos 23 padrões catalogados no livro de Eric Gamma e de seus colaboradores. O problema que esse padrão considera é o seguinte: como definir uma relação hierárquica entre objetos de tal forma que tanto o objeto todo quanto os objetos parte sejam equivalentes em certos aspectos?

O padrão Composite pode ser utilizado para solucionar o problema de representar uma hierarquia de composição recursiva entre entidades. Esse problema é conhecido como "problema da explosão de partes" (*parts explosion problem*). O exemplo clássico desse problema é o da montagem de peças: uma peça é composta de diversas outras peças, e essas peças componentes são elas próprias compostas, e assim por diante.

A Figura 8-26 exibe a estrutura do padrão Composite (são apresentadas somente as classes, sem atributos nem operações). O Componente é uma superclasse. Há dois tipos de componente, Folha e Composto. Uma folha não possui componentes, enquanto um composto possui. A associação entre Composto e Componente é que representa a hierarquia de composição recursiva: um composto possui diversos componentes, que podem ser ou folhas ou outros compostos.

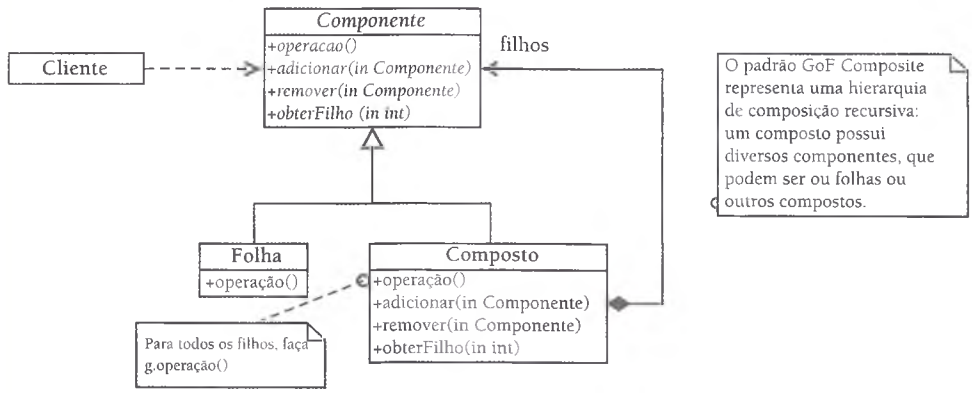


Figura 8-26: O Padrão Composite.

8.6.2 Observer

O objetivo desse padrão é definir de forma flexível uma dependência um para muitos entre objetos. Ou seja, o padrão Observer é útil quando há um objeto central e diversos outros objetos que *dependem* do primeiro. A dependência aqui é no sentido de que, se houver alguma modificação no estado do objeto central, os objetos dependentes devem ser notificados. A preocupação aqui é com respeito ao acoplamento: necessitamos que o objeto central seja capaz de enviar mensagens de notificação aos seus dependentes sem, no entanto, conhecê-los diretamente. Em outras palavras, desejamos que haja um acoplamento fraco entre os objetos dependentes e o objeto central.

A solução que o padrão Observer descreve para o problema é fazer com que os objetos dependentes implementem uma interface (ver Seção 8.5.4) comum para propiciar o acoplamento abstrato entre cada dependente e o objeto central. Além disso, o objeto central deve manter uma lista de referências para os seus dependentes. Quando o estado do objeto central é modificado, os dependentes são comunicados por intermédio dessa interface. Tudo o que o objeto central precisa saber é que existem objetos que implementam essa interface. No entanto, o objeto central não precisa ter conhecimento de quais são as classes dos objetos dependentes.

Quadro 8-8: Interface Observer

```
interface Observer {
   void update(Observable t, Object o);
}
```

A interface que o objeto central assume que seus dependentes implementam é apresentada no Quadro 8.8 (utilizamos a sintaxe da linguagem Java). Essa interface possui uma única operação, *update*. A classe Observable é uma superclasse do objeto central. Ao ser modificado, esse objeto notifica aos seus dependentes chamando a operação update e passando a si próprio como primeiro argumento. Dessa forma, cada observador (objeto dependente) pode consultar o objeto central para saber o que foi modificado neste último.

A razão para a necessidade de a classe do objeto central ser uma subclasse de Observable é que essa última fornece duas funcionalidades importantes para suas subclasses: (1) a de *notificação* dos dependentes e (2) a de *manutenção* desses dependentes. Em particular, a classe Observable em Java contém as seguintes operações, para manutenção da lista de objetos dependentes:

```
public void addObserver(Observer obs)
Adiciona um novo objeto na lista de objetos dependentes.

public void deleteObserver(Observer obs)
Remove um objeto da lista de objetos dependentes.
```

Note que essas operações são definidas em termos da interface Observer. Isso significa que, para um objeto ser inserido na lista de dependentes (observadores) do objeto central, basta o primeiro implementar a interface Observer. A Figura 8-27 dá uma visão geral da estrutura do padrão Observer.

Uma aplicação prática do padrão de projeto Observer é na implementação da comunicação entre camadas de software (ver Seção 11.1.1). O padrão Obser-

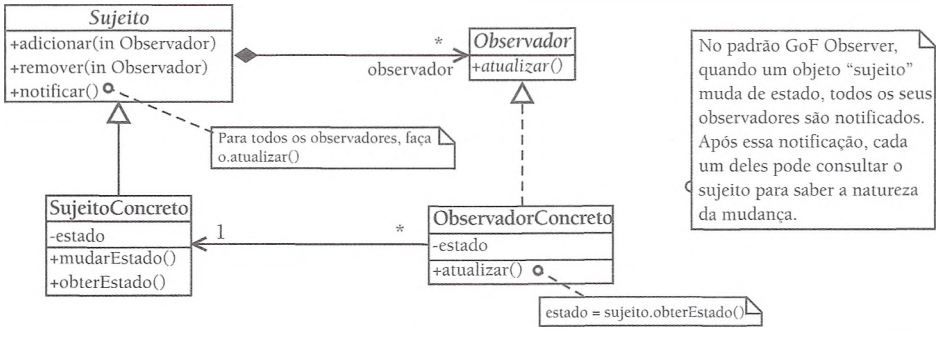


Figura 8-27: Estrutura do padrão Observer.

ver é útil nesse caso para permitir que uma camada de software mais genérica possa enviar sinais para uma camada menos genérica. O elemento pertencente à camada mais genérica, representado pelo objeto central, pode estar associado a diversos objetos que dele dependem e que estão localizados na camada mais específica, que são representados como objetos dependentes.

8.6.3 Strategy

O padrão Strategy tem o objetivo de encapsular diferentes algoritmos para realização de alguma tarefa computacional por trás de uma interface e permitir que a região de código cliente dessa tarefa possa utilizar qualquer desses algoritmos sem precisar ser modificada. Podemos também interpretar o padrão Strategy como uma forma de desacoplar uma região de código cliente de uma tarefa das diferentes maneiras de implementar essa tarefa.

Como exemplo do padrão Strategy, considere novamente nosso estudo de caso representado pelo Sistema de Controle Acadêmico (ver Seção 4.7). A instituição de ensino na qual o SCA está para ser implantado utiliza uma forma de calcular o grau final de um aluno em uma disciplina cursada. Esse grau é uma letra: A, B, C, D ou E. Além disso, esse grau é calculado a partir de notas atribuídas a avaliações. O projetista do SCA identificou que, atualmente, cada nota varia na faixa de 0 a 10. No entanto, identificou que é comum a coordenação modificar a estratégia (algoritmo) de atribuição de graus a partir da notas das avaliações. Esse projetista definiu o diagrama de classes da Figura 8-28, correspondente a uma estrutura de classes para cálculo do grau. Nesse diagrama, nessa solução, *Participação* é uma classe que representa participações de alunos em uma disciplina e suas correspondentes avaliações em termos de notas. Além disso, a operação *calcular* retorna o grau do aluno.

A solução proposta pelo projetista foi inspirada pelo padrão de projeto Strategy. Na Figura 8-28, temos uma hierarquia de classes. A raiz dessa hierarquia é uma classe abstrata. Essa classe possui uma operação abstrata, *calcular*. As duas

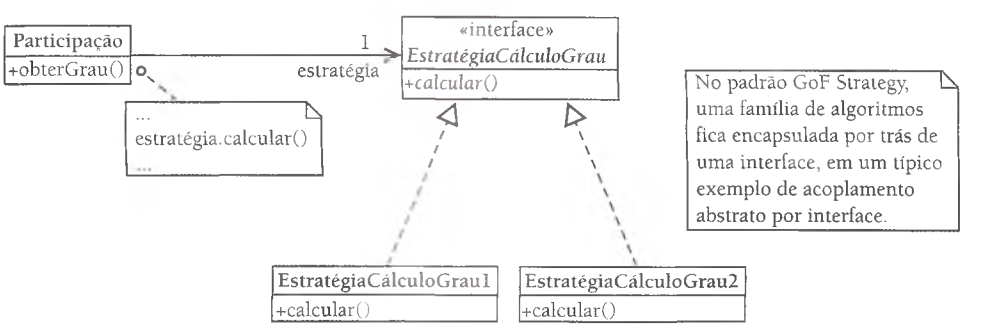


Figura 8-28: Exemplo do padrão de projeto Strategy.

subclasses fornecem implementação para a operação abstrata que herdam de sua superclasse. Uma região de código da aplicação fica responsável (1) por identificar qual é a estratégia vigente para cálculo de grau e (2) por instanciar o objeto da subclasse correspondente. Uma vez que esse objeto está instanciado, a região de código cliente pode, por intermédio da interface, enviar a mensagem que ativa a execução do método calcular.

A grande vantagem dessa solução está na flexibilidade resultante. Para entender isso, note que a região de código cliente não precisa ser alterada quando a forma de cálculo de grau tiver que ser alterada. Tudo o que a região de código cliente conhece acerca da região de código fornecedora do serviço (nesse caso, o cálculo do grau do aluno em uma disciplina) é que existe uma operação chamada calcular, que realiza o cálculo. A região de código cliente não sabe nem precisa saber qual o objeto específico que está fornecendo uma implementação dessa operação.

É importante notar também que uma solução alternativa equivalente à utilização de uma classe abstrata seria utilizar uma *interface* (ver Seção 8.5.4). Nessa solução, as classes `EstratégiaCálculoGrau1` e `EstratégiaCálculoGrau2` iriam implementar a interface. Em ambos os casos, entretanto, o *acoplamento abstrato* (ver Seção 8.5.5) é mantido entre os elementos envolvidos.

8.6.4 Factory Method

Quando uma determinada parte de um SSOO precisa dos serviços de um objeto, essa parte envia mensagem para este último. No entanto para que um objeto receba mensagens, ele deve existir: não faz o menor sentido falar em "envio de mensagens" para um objeto que não existe. Para um objeto existir, ele deve ser *instanciado* a partir de uma classe. Instanciar uma classe significa criar um objeto dessa classe. Fisicamente falando, isso significa (1) alocar espaço em memória para armazenar o objeto e (2) iniciar o seu estado (ou seja, os valores de seus atributos). Após a instanciação, o objeto pode prover serviços para outros objetos.

A operação de instanciação de um objeto pode ser bastante complexa. Além disso, pode ser que não seja adequado fazer com que uma região de código que necessite de um serviço tenha uma referência direta para a classe que o fornece. Uma razão para isso pode ser o fato de que a forma de implementar tal serviço é instável, no sentido de precisar ser alterada no futuro. Se a região de código instanciar diretamente a classe que lhe fornece determinado serviço, essa região fica definitivamente acoplada a essa classe fornecedora e a sua forma específica de prover o serviço requerido.

Uma forma de resolver o problema descrito no parágrafo anterior é com uma *fábrica de objetos*. Uma fábrica de objeto corresponde a uma classe cuja res-

ponsabilidade é criar quando requisitada. O fato é que essa fábrica retorna o objeto criado na forma de uma referência para uma *superclasse abstrata* (ver Seção 8.5.2) ou para uma *interface* (ver Seção 8.5.4). Dessa maneira, a região cliente, que previamente criava o objeto para lhe prover o serviço, agora fica dependente desse serviço através de um *acoplamento abstrato* (ver Seção 8.5.5). Com efeito, a classe que realmente fornece o serviço requerido pode ser alterada sem que a região de código cliente precise de modificação. Em essência, essa é a solução que o padrão Factory Method descreve. Em outras palavras, o objetivo do padrão Factory Method é definir uma interface para instanciar objetos. Esse é um dos padrões criacionais definidos pelo GoF.

Note que a fábrica de objetos precisa fazer referência às classes concretas cujos objetos fornecem o serviço requerido, o que é um exemplo de *acoplamento concreto*. Entretanto, essa referência fica localizada em uma região de código particular (na fábrica de objeto) e é conseqüentemente aceitável. Note que quaisquer mudanças nas classes que fornecem o serviço em questão ficam localizadas na fábrica de objetos e não afetam os clientes.

Outro aspecto importante sobre o padrão Factory Method é que sua utilização adiciona complexidade ao projeto. Na verdade, essa é uma tônica na maioria dos padrões de projeto: eles adicionam *flexibilidade* à custa de adicionarem *complexidade* à solução. No caso particular do padrão Factory Method, a decisão por utilizar o mesmo deve levar em consideração o fato de a classe que fornece o serviço ser realmente suscetível a mudanças futuras. Do contrário, a referência (instanciação) direta da classe que fornece o serviço (em vez de utilização do padrão Factory Method) é uma alternativa mais viável.

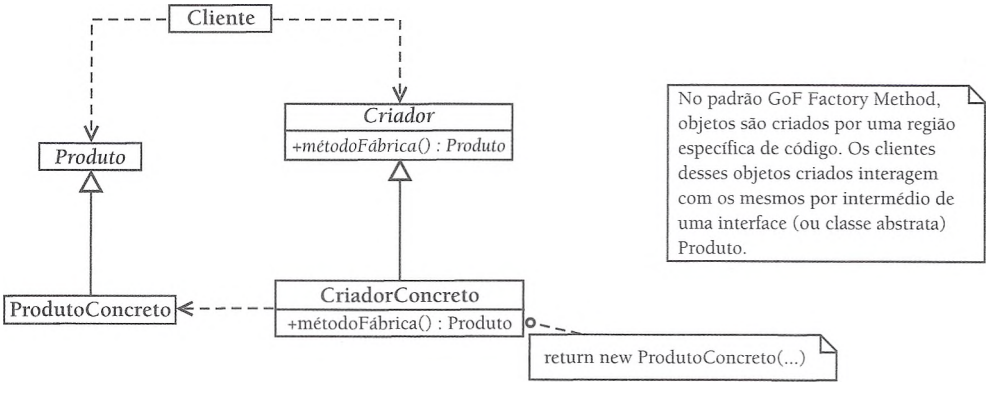


Figura 8-29: Estrutura do padrão Factory Method.

Outro padrão GoF de criação comumente utilizado é o chamado *Abstract Factory*. Esse padrão é uma extensão do Factory Method para criação de uma família de objetos relacionados.

8.6.5 Mediator

O padrão de projeto Mediator permite a um grupo de objetos interagir, ao mesmo tempo que mantém um acoplamento fraco entre os componentes desse grupo. A solução proposta pelo padrão Mediator para alcançar esse objetivo é definir um objeto, o *mediador*, para encapsular interações da seguinte forma: o resultado da interação de um subgrupo de objeto é passado a outro subgrupo pelo mediador. Dessa forma, os subgrupos não precisam ter conhecimento da existência um do outro e podem variar independentemente. Os objetos de controle, cuja especificação descrevemos na Seção 8.1.3, são exemplos de mediadores.

8.6.6 Façade

No Capítulo 11, descrevemos aspectos relativos à divisão de um SSOO em subsistemas. Nessa divisão, é necessário definir as interfaces de comunicação (ou interação) entre os subsistemas resultantes. Nesse contexto, quando dois subsistemas se comunicam, podemos dizer que há um cliente e um fornecedor. O subsistema cliente requisita algum serviço, e o subsistema fornecedor é o que provê esse serviço. O padrão Façade procura resolver o seguinte problema: *como definir uma interface de alto nível que torna um subsistema mais fácil de ser utilizado*? Em outras palavras, de que maneira podemos definir uma interface de comunicação mínima possível entre os subsistemas cliente e fornecedor?

A solução fornecida por este padrão é a seguinte: criar uma *fachada* para o subsistema fornecedor, de tal forma que o subsistema cliente se comunique com o primeiro por intermédio desta fachada. A vantagem dessa solução é que o

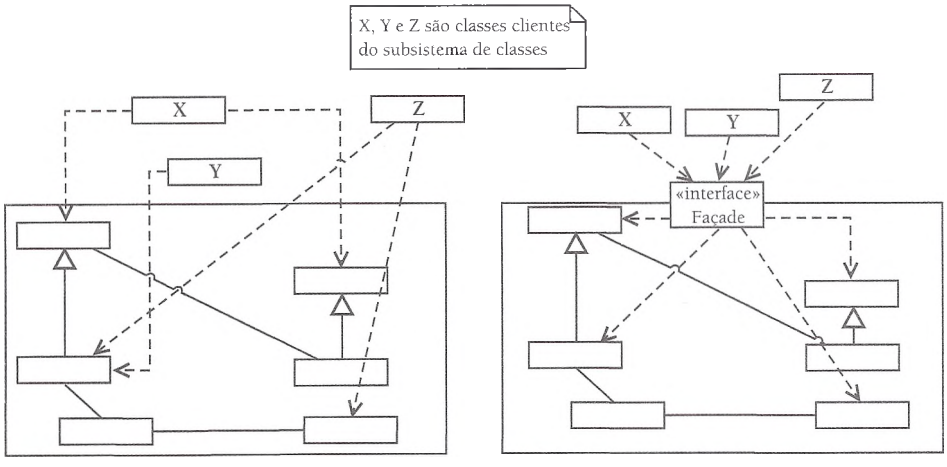


Figura 8-30: Estrutura do padrão Façade.

cliente somente conhece o necessário e suficiente em relação à complexidade do fornecedor. Toda a complexidade adicional desse último fica "escondida" por trás da interface de comunicação.

A implementação do padrão Façade é relativamente simples. Consiste em definir uma ou mais classes que implementam a interface de comunicação necessária e suficiente. O subsistema fornecedor fica encapsulado por essa interface.

8.7 Modelo de classes de projeto em um processo iterativo

Em um processo de desenvolvimento iterativo, o modelo de classes de projeto é construído em diversas iterações. Normalmente, esse modelo é construído em paralelo com o *modelo de interações* (ver Capítulo 7). Isso porque o modelo de interações fornece informações para completar o modelo de classes em seu estágio de análise a fim de levá-lo ao estágio de projeto. Em particular, uma mensagem que identificamos durante a construção dos diagramas de interações sempre corresponde a uma operação na classe do objeto remetente. Para cada mensagem identificada, a assinatura da operação correspondente deve ser completamente especificada no modelo de classes de projeto.

Conforme descrevemos na Seção 10.2.3, o diagrama de atividades pode ser utilizado na construção do modelo de classe de projeto, com o objetivo de especificar alguma operação um tanto mais complexa. No entanto, é importante notar que, em um SSOO, operações são normalmente simples, o que dispensa o detalhamento de sua lógica interna. No mais das vezes, essas operações têm 10 instruções ou menos. Algumas vezes, essas operações têm uma ou duas instruções. Além disso, o modelo de interações fornece informações suficientes para a implementação correta da lógica de funcionamento de uma operação. Em suma, responsabilidades e operações que não são nem simples nem claras sugerem que a classe correspondente está mal projetada. Da mesma forma, classes com uma quantidade excessiva de operações (mais de uma dúzia) também são altamente suspeitas. Aliás, clareza e simplicidade devem ser perseguidas desde a fase de análise, durante a identificação das classes iniciais. Lembre-se da modelagem CRC, em que cada cartão possui um tamanho fixo, o que força o modelador a projetar classes simples. Os mesmos princípios de clareza e simplicidade também devem ser perseguidos durante a modelagem de interações, pois essa atividade gera informações para a completa especificação das operações.

O conceito de reuso é fundamental durante a construção do modelo de classes de projeto. Os padrões de projeto, para os quais demos uma pequena introdução na Seção 8.6, são mecanismos de reuso de soluções para problemas recorrentes. Outra forma de reuso que deve ser considerada na fase de projeto,

não somente para a modelagem de classes, é o uso de bibliotecas de classes e de *frameworks*. Na Seção 12.2.4, descrevemos em mais detalhes o conceito de framework em um contexto específico, o da *persistência de objetos*.

É adequado dizer que as atividades de especificação (projeto) e de implementação são intimamente entrelaçadas. Nas palavras de Philippe Krutchen, um reconhecido arquiteto de software: "Eu codifico para melhor entender o que estou projetando." Essa sinergia entre o projeto e codificação de um SSOO se justifica pelo fato de que, ao ser forçado a moldar certa decisão de projeto em uma implementação concreta e sem ambigüidades, o desenvolvedor valida a completa utilidade das classes que projetou. Note outra declaração de dois outros respeitados tecnologistas, Don Roberts e Ralph Johnson: "As pessoas desenvolvem abstrações através da generalização aplicada a exemplos concretos. Cada tentativa de determinar abstrações corretas no papel, sem realmente desenvolver um sistema, está fadada ao insucesso." Ainda outra declaração, dessa vez do lendário Fred Brooks: "Planeje para jogar um [versão do sistema] fora; isso acontecerá de qualquer maneira." Todas essas declarações carregam a mesma mensagem, em essência: quando uma parte do sistema é implementada, isso resulta em informações para melhorar a qualidade do projeto do qual partiu a implementação. Nesse contexto, a construção de *protótipos* (ver Seção 2.5) para validar as decisões de projeto é importante.

Uma dúvida freqüente diz respeito ao nível de detalhe na especificação dos diagramas de classes de projeto. Há diversos fatores que influenciam a resposta para essa questão. Em primeiro lugar, devemos perceber que um modelo que não esteja em conformidade com o sistema não tem valia. Por outro lado, quanto mais detalhados os modelos que construímos possuem, mais difícil mantê-los em sincronia com o código-fonte, pois este último é muito instável. Se a equipe de desenvolvimento tiver à disposição ferramentas CASE que permitam realizar engenharias direta e reversa (ver Seção 2.6), a construção de modelos mais detalhados é possível. Isso porque as próprias ferramentas podem atualizar os modelos existentes. Entretanto, se este não for o caso, a equipe corre o risco de perder tanto tempo mantendo os modelos atualizados quanto leva para realmente implementar o sistema. Portanto, o bom senso também vale para decidir o nível de detalhamento que deve ser adotado na construção, não só dos diagramas de classes, mas de qualquer modelo de um SSOO.

8.8 Estudo de caso

Para exemplificar a especificação de classes no estudo de caso do Sistema de Controle Acadêmico, considere o diagrama de classes correspondente à visão de classes participantes do caso de uso Realizar Inscrição. A Figura 8-31 apresenta uma versão inicial do refinamento desse diagrama, considerando vários

assuntos tratados neste capítulo. Por simplificação, os parâmetros das operações não são exibidos.

Note que, por simplificação, apenas o refinamento correspondente a uma das visões de classes participantes é apresentado. Em uma situação real, todas as classes de análise devem ser consideradas..

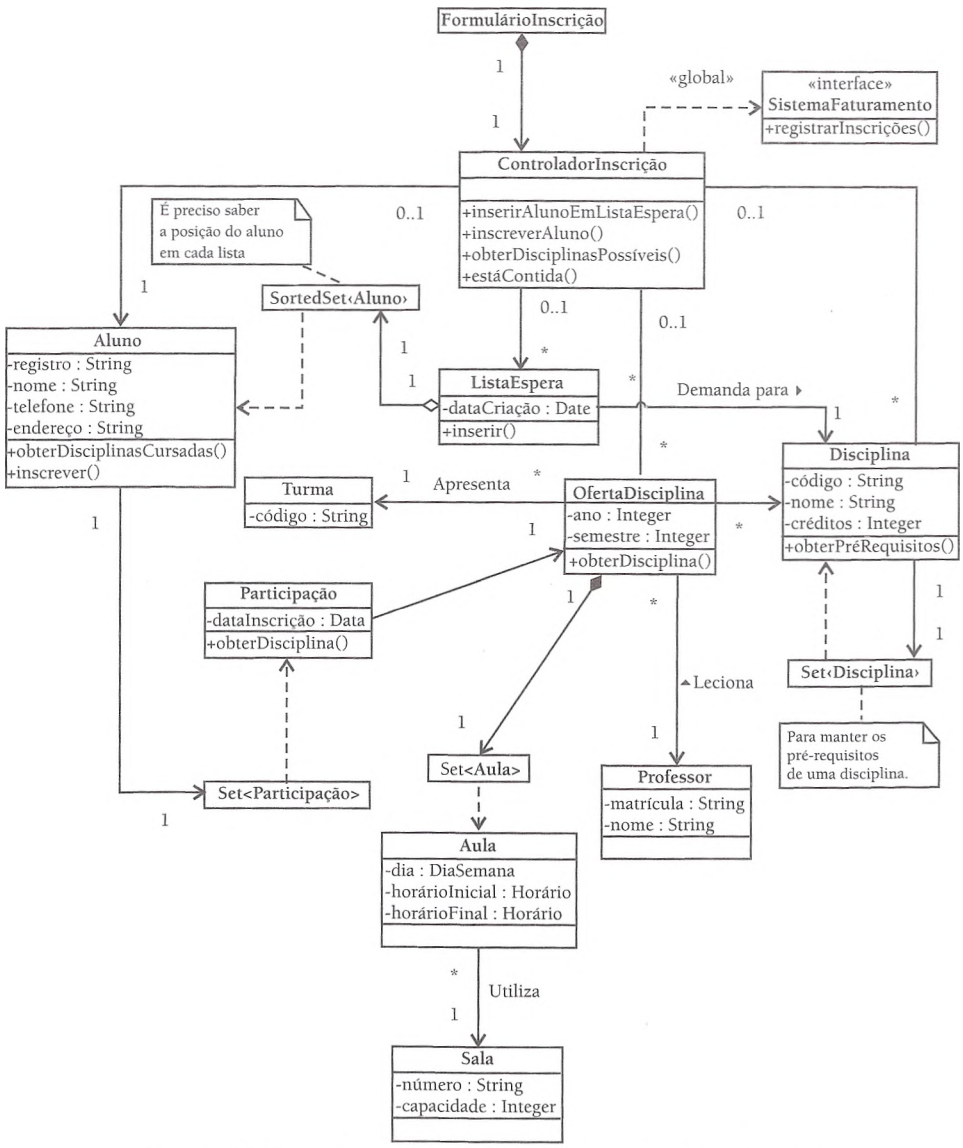


Figura 8-31: Especificação da VCP do caso de uso Realizar Inscrição.

EXERCÍCIOS

8-1: No Capítulo 7, descrevemos os conceitos de coesão e acoplamento. Qual é a relação desses conceitos com a qualidade resultante da modelagem de classes de projeto?

8-2: Este capítulo apresenta o relacionamento de dependência e descreve diversos tipos de dependência que podem existir (por atributo, por variável local etc.). Seja uma classe A que possui pelo menos uma operação cujo tipo de retorno é a classe B. Que tipo(s) de dependência pode haver entre A e B?

8-3: Em cada um dos itens a seguir, discuta qual tipo de relacionamento é mais adequado (associação, agregação, composição). Desenhe o diagrama de classes correspondente, indicando as multiplicidades. Especifique, ainda, atributos e possíveis restrições.

a) Um País possui uma Capital.
b) Uma Empresa é subsidiária de diversas outras Empresas.
c) Uma Pessoa à mesa de jantar está usando uma Faca.
d) Um Polígono é composto por um conjunto ordenado de Segmentos.
e) Um Estudante acompanha uma Disciplina com um Professor.
f) Uma Caixa contém Garrafas.
g) Um Livro contém Capítulos.
h) Um Arquivo contém Registros.

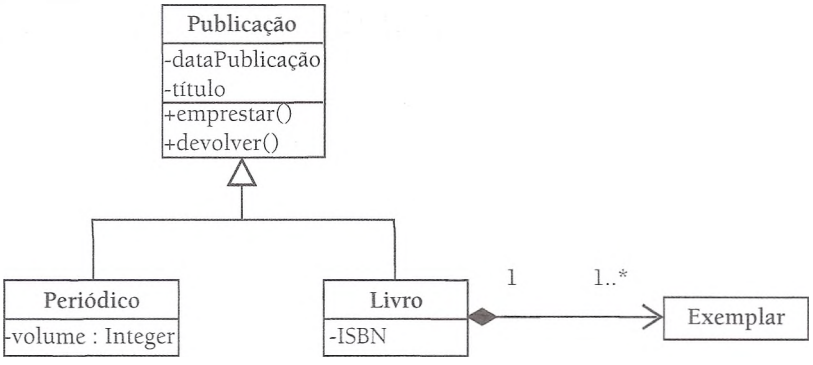
8-4: Para cada um dos itens do exercício anterior, defina refinamentos sobre as associações existentes, considerando diversas possibilidades de navegabilidade entre tais associações.

8-5: Forme uma hierarquia de classes a partir dos seguintes conceitos: Pessoa, Empregado, Instrutor e Estagiário.

8-6: Em cada um dos itens abaixo, desenhe o diagrama de classes correspondente, indicando as multiplicidades. Especifique, ainda, possíveis restrições que se apliquem.

a) Uma Pessoa, como programador, utiliza uma Linguagem de Programação.
b) Um Objeto de Desenho pode ser um Texto, um Gráfico ou um Grupo de objetos.
c) Modem, Teclado e Impressora são Dispositivos de Entrada e Saída.
d) Um Banco de Dados contém Tabelas de Sistema e Tabelas de Usuário. Uma Tabela de Sistema mantém informações sobre uma ou várias Tabelas de Usuário. Uma Tabela contém Registros.
e) Um Item pode ser um Item Atômico ou um Item Composto. Um Item Composto possui dois ou mais Itens.

8-7: O que há de errado com o diagrama de classes a seguir? Construa uma nova versão deste diagrama, consertando os erros identificados.



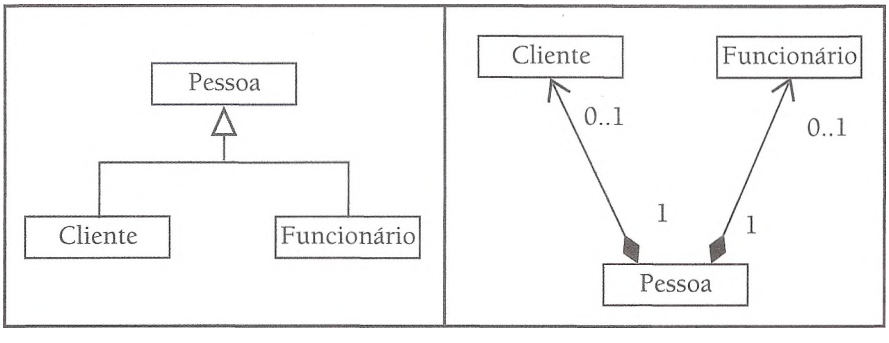
8-8: Crie um diagrama de classes de projeto para representar os conceitos de círculo e de elipse como classes. Defina uma operação desenhar em ambas as classes. Dica: crie uma superclasse abstrata para organizar a hierarquia.

8-9: Considere as classes Rádio e Relógio a seguir. Qual o problema em definir uma classe RádioRelógio como subclasse das duas classes (herança múltipla)? Defina uma solução, utilizando o mecanismo de delegação, para substituir a solução por herança múltipla. Defina outra solução através de interfaces.

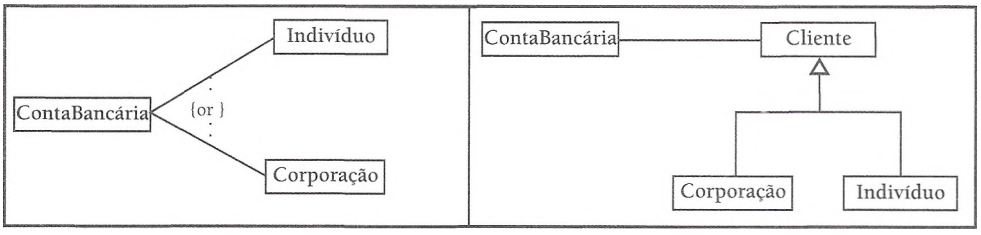
Rádio
-estação
+ligar +desligar +obterEstação() +definirEstação()

Relógio
-horário
+ligar +desligar +obterHorário() +definirHorário()

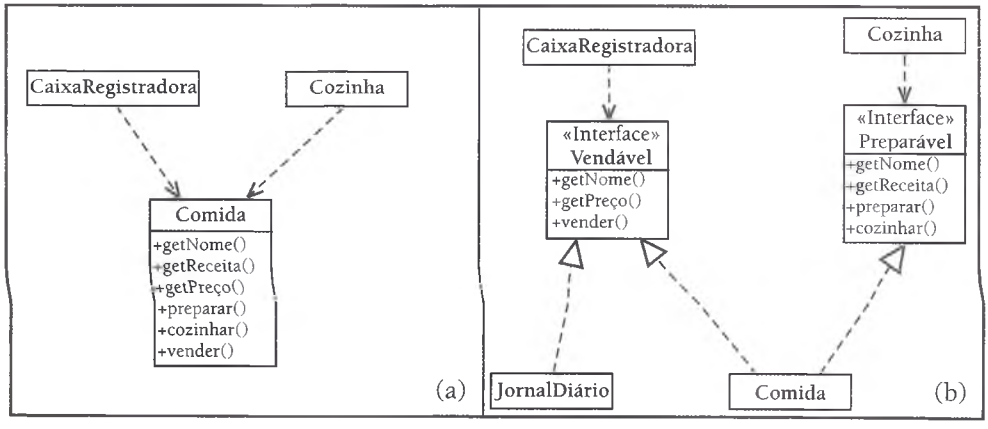
8-10: No desenvolvimento de um sistema de software comercial, um modelador precisou representar as classes clientes e funcionários de uma empresa. Em vez de optar pela solução do diagrama à esquerda, ele optou pela solução representada à direita na figura a seguir. Discuta as vantagens e desvantagens de cada uma das duas soluções. Reflita sobre a situação em que um cliente pode também ser um funcionário da empresa. E se uma classe gerente tivesse de ser modelada?



8-11: Considere os diagramas de classes de análise fornecidos nos itens (a) e (b) a seguir, ambos de acordo com a notação da UML. Esses diagramas desejam representar o fato de que uma conta bancária pode estar associada a uma pessoa, que pode ser ou uma pessoa física (representada pela classe *Indivíduo*), ou uma pessoa jurídica (representada pala classe *Corporação*). Uma dessas duas soluções é melhor que a outra? Se sim, qual delas e em que sentido?



8-12: Considere uma aplicação para um bar-café. Nessa aplicação, considere a existência de uma classe que representa um comestível qualquer vendido pelo bar-café: *Comida*. Considere ainda duas outras classes nessa aplicação, *Cozinha* e *CaixaRegistradora*. A classe cozinha manipula objeto da classe Comida para montar pratos. Já a classe CaixaRegistradora manipula objeto comida para registrar a venda dos mesmos e cobrar por eles. Portanto, essas duas classes dependem dos serviços fornecidos pela classe Comida. Em um primeiro modelo dessa aplicação, o modelador fez com que as classes Cozinha e CaixaRegistradora dependessem diretamente da classe Comida, conforme a Figura (a). No entanto, conforme o desenvolvimento foi se evoluindo, o modelador identificou um novo requisito na aplicação: agora era preciso registrar a venda de coisas não comestíveis. Por exemplo: o café-bar passou a vender jornais diários. Para atender ao novo requisito, o modelador criou duas novas interfaces, Vendável e Comestível, conforme está na Figura (b). Discuta a decisão de projeto do modelador. Você achou a decisão adequada? Que princípios de projeto levaram o modelador a tomar tal decisão?



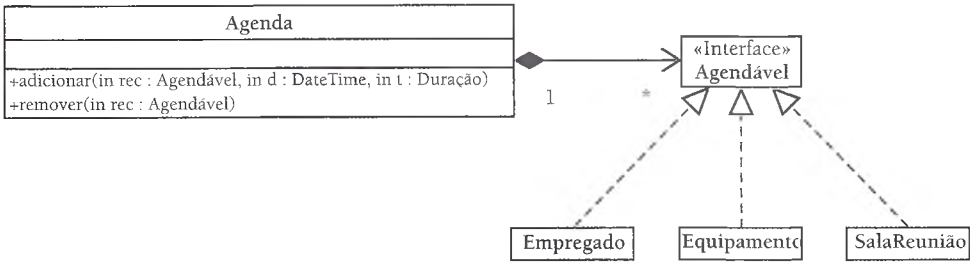
8-13: Você está desenvolvendo uma aplicação para uma empresa que vende componentes de computador. Atualmente, você está construindo uma hierarquia de classes para representar os diferentes tipos de componentes (você nomeou essas classes como *Processador, DiscoRígido*

e *CDROM*). Essa hierarquia contém também uma superclasse abstrata denominada *Componente*, da qual são derivadas as demais classes. Por outro lado, um amigo seu já desenvolveu um sistema semelhante e lhe passou classes para representar discos rígidos e CPUs (as classes dele são denominadas *HardDisk* e *CPU* respectivamente). De que maneira você pode construir sua hierarquia de classes aproveitando (reutilizando) as classes fornecidas pelo seu amigo, sem precisar modificá-las? Que padrão de software poderia ser empregado para auxiliar na definição da estrutura de classes a fim de representar a situação discutida anteriormente? Utilizando esse padrão, forneça um fragmento de diagrama de classes que represente essa situação.

8-14: Suponha que você precise criar em uma aplicação um objeto que, uma vez instanciado, não deve ter seu estado alterado sob hipótese algum. Descreva uma solução para esse problema. Pode pensar em possíveis aplicações para essa situação. Dica: procure por *Immutable Pattern* na Internet.

8-15: Suponha que você precise ter uma classe C em sua aplicação para a qual a forma de acesso é diferenciada, no seguinte sentido: há um conjunto de classes que têm permissão para alterar o estado dos objetos da classe C; há também outro conjunto de classes que, embora façam uso dos serviços fornecidos por objetos da classe C, não têm permissão para modificar o estado desses objetos. Defina uma solução de projeto para esse problema. Dica: utilize o conceito de interface.

8-16: Imagine que um projetista está especificando um SSOO para gerenciar conferências. Uma conferência possui diversos recursos cuja alocação precisa ser agendada. Ou seja, para certo evento em uma conferência, diversos recursos são alocados. Os recursos são de diversas naturezas: funcionários, equipamentos, salas de apresentação. Suponha que você é esse projetista. Que solução de projeto poderia ser definida para esse problema?



8-17: Analise as relações existentes entre o conceito de interface e os princípios de polimorfismo e de encapsulamento da orientação a objetos.

8-18: Defina as diferenças e semelhanças existentes entre uma classe abstrata e uma interface. Qual é a diferença entre uma classe concreta que herda de uma classe abstrata e a realização de uma interface?